

Enfoque de detección de tendencias de valores basado en la observación de las repercusiones en redes sociales

Tesis de Grado de la Carrera de Ingeniería en Sistemas de la Universidad
del Centro de la Provincia de Buenos Aires



Directores: Dr. Armentano Marcelo

Dr. Berdun Franco

Alumno: Ojeda Juan Ezequiel

1.1 Motivación	3
1.2 Objetivos	4
2. Marco Teórico	7
2.1 Qué es el Bitcoin y cómo funciona	7
2.2 Minería de datos	8
2.2.1 Redes Bayesianas	11
2.2.2 Algoritmos utilizados	12
Decision Table	12
Naive Bayes	13
SMO	13
J48	13
SGD	14
2.3 Aprendizaje de máquina	14
2.4 Procesamiento de Lenguaje Natural	15
2.5 Tendencias en Twitter	16
2.7 Trabajos relacionados	17
3. Desarrollo de la propuesta	19
3.1. Estructura de la herramienta desarrollada	19
Convertir archivo Json a arff	20
Crear clasificador a partir de un archivo arff	21
Clasificar archivos en formato JSON	22
3.2. Enfoque de clasificación de propuesto	22
3.2.1 Enfoque propuesto	22
3.2.2 Pasos previos	22
4. Diseño e implementación de la herramienta propuesta	24
4.1 Herramientas utilizadas	24
4.1.1 Twitter4J	24
4.1.2 Google Natural Language (GNL)	25
4.1.3 WEKA	25
4.2. Requerimientos funcionales y restricciones de diseño	34
4.3 Diseño de la solución	29
4.4 Escenarios de calidad	34
4.4.1 Usabilidad	34
4.4.2 Fiabilidad	31
4.4.3 Precisión	34
4.4.4 Performance	33

4.4.5 Portabilidad	34
4.4.6 Modificabilidad	34
4.5 Persistencia de los datos	34
4.6 Resumen	34
5. Diseño e implementación de la aplicación de análisis de datos	35
5.1 Requisitos funcionales y atributos de calidad	36
5.2 Decisiones de diseño	38
5.2.1 Capa de presentación	38
5.2.2 Capa de Negocio	39
5.3 Atributos de calidad	40
5.3.1 Usabilidad	40
5.3.2 Portabilidad	46
5.3.3 Disponibilidad	47
5.3.4 Escalabilidad	47
5.4 Persistencia de datos	48
6. Descripción del experimento	49
6.1. Configuración	49
6.2. Resultados de experimentación con técnicas de aprendizaje de máquina	51
6.2.1 Análisis general clasificadores	53
6.2.2 Análisis individual clasificadores entrenamiento	54
6.3 Aplicación y evaluación de la herramienta resultante	55
7. Conclusiones	68
7.1 Limitaciones	68
7.2 Trabajos futuros	69
7.3 Implicancias	69
8. Anexos	70
8.1 Comerciantes de ruido	70
8.2 EC2	71
8.3 Putty	72
8.4 Listener	72
Referencias	73

1.Introducción

El objetivo general de esta investigación es estudiar si el tipo de cambio de bitcoin refleja las opiniones de las personas sobre la moneda digital. Para explorar esto, es importante establecer si la investigación ha demostrado que los inversionistas se ven afectados por el sentimiento en relación con otras inversiones tradicionales como las acciones. También es oportuno examinar las características de las inversiones más propensas a los comentarios de la gente, y si esas mismas características se aplican a bitcoin. Además, dado que este documento utiliza datos públicos disponibles de Internet, es importante establecer la confiabilidad de esa fuente, dónde se ha utilizado anteriormente, particularmente en relación con los mercados financieros.

1.1 Motivación

Actualmente los canales de internet (como blogs, tweets, foros, chats, correo electrónico, etc.) son ampliamente utilizados como medios de comunicación. Los datos de las redes sociales representan un indicador colectivo de pensamientos e ideas con respecto a cada aspecto del mundo. Ha sido posible asistir a cambios profundos en los hábitos de las personas en el uso de redes sociales como Twitter que se ha convertido en una herramienta importante para las empresas e individuos para comunicarse y compartir información con un rápido crecimiento y adopción significativa. Además, Twitter ha crecido rápidamente como un medio para compartir ideas y pensamientos sobre decisiones de inversión.

Asumiendo esto, podemos decir que los usuarios de internet juegan un rol relevante sobre las tendencias, por ejemplo en la economía. Un claro ejemplo es el del dólar. Cuando muchas personas piensan que el dólar va a subir, comienzan a comprar y en consecuencia el valor del dólar aumenta (Ron D. et al., 2013). Ocurre análogamente con el caso de la baja. Otro caso que ganó interés en el último tiempo es el de la criptomoneda Bitcoin. Debido a sus propiedades, Bitcoin no se gestiona por ningún gobierno o banco. Como cualquier otra moneda, una peculiaridad de Bitcoin es facilitar las transacciones de bienes y servicios, atrayendo un gran número de usuarios y mucha atención mediática.

El análisis de sentimiento (sentiment analysis), también conocido como minería de opinión (opinion mining), es un área de investigación relativamente reciente que intenta determinar el tono emocional que hay detrás de una serie de palabras, y se utiliza para intentar entender las actitudes, opiniones y emociones expresadas en un determinado texto, generalmente escrito. El análisis de sentimiento es extremadamente útil en la monitorización de las redes sociales ya que permite obtener una idea de la opinión pública general sobre ciertos temas (Prieto-Gutierrez, 2011).

En este trabajo se decidió aplicar el análisis de sentimiento automatizado en mensajes cortos compartidos de usuarios en Twitter para analizar automáticamente las opiniones,

sentimientos, evaluaciones y actitudes de las personas. Se investiga si el sentimiento público, como el expresado en colecciones a gran escala de publicaciones diarias de Twitter, puede usarse para predecir la tendencia del mercado de Bitcoin. Con esto, se busca descubrir si las discusiones de la comunidad pueden ser utilizadas para hacer predicciones cualitativas sobre el mercado de Bitcoin, tratando de establecer si hay alguna correlación entre el sentimiento del tweet y el Bitcoin.

Un premisa a tener en cuenta al momento de buscar una solución a la detección de la tendencia es el rol que juegan los usuarios a la hora de realizar publicaciones teniendo en cuenta sus características (popularidad, cuenta verificada, etc) sabiendo que dichas características podrían darle mayor o menor ponderación a su publicación.

La hipótesis seguida en este trabajo de tesis es que el análisis de redes sociales facilita la detección de tendencias en el mercado de valores. Como caso de estudio se optó por el Bitcoin ya que es una opción con gran cantidad de material disponible en las redes sociales para realizar el análisis.

1.2 Objetivos

La propuesta del trabajo consiste en el desarrollo de enfoques para la predicción de tendencias en las divisas, para lo que se aplicarán técnicas de Aprendizaje de Máquina, ya que brinda la posibilidad de aprender de los datos para que el software evolucione automáticamente.

Esta propuesta está basada en el objetivo general del trabajo, el cual consiste en desarrollar una herramienta que provea la asistencia necesaria para poder realizar especulaciones en las tendencias monetarias, utilizando al Bitcoin como caso de estudio.

La herramienta a desarrollar utilizará métricas de confianza en las publicaciones consideradas analizando características de los usuarios (si la cuenta está verificada, cantidad de seguidores, ubicación geográfica, etc), características de su publicación (palabras clave, análisis de sentimiento, comentario positivo o negativo, etc).

Una vez realizado el análisis, se les dará determinado puntaje a la publicación (un tweet) para llevar a cabo la especulación de la tendencia.

Para cumplir con los objetivos se deberán desarrollar en el trabajo las siguientes funcionalidades:

- **Entrenamiento:** En esta etapa se aplicará Aprendizaje de Máquina sobre un dataset pre-procesado. Dicho dataset fue capturado del stream de Twitter durante un periodo de tres meses (corriendo con un servidor de Amazon y filtrando por palabras claves). Este dataset fue etiquetado durante intervalos de un día indicando si el Bitcoin subió realmente o bajó en cada intervalo.

- Clasificación: Luego del entrenamiento se procederá a la clasificación de un conjunto de datos de entrada no clasificados, utilizando algoritmos de Aprendizaje de Máquina.
- Análisis de los tweets: Se analizará la información recolectada de Twitter a través de la herramienta entrenada. El análisis se enfoca en obtener un resultado aceptable en la clasificación de tendencias sobre la ventana de tiempo de tres meses de dataset.
- Especulación: En esta etapa se tendrá en cuenta qué tipo de tendencia se detectó (tendencia a la alta o a la baja) con el objetivo de predecir dicha tendencia.
- Generación de resultados: Se generarán resultados a partir de la información recolectada. Estos resultados son la salida final de la herramienta, reflejando el resultado de la tendencia.

El trabajo a seguir en esta tesis puede dividirse en dos grandes etapas: Diseño e implementación de la herramienta y Análisis de los datos.

Durante el diseño e implementación de la herramienta se debe diseñar e implementar la misma para clasificar las tendencias en las divisas. Esta herramienta recibirá un archivo para entrenar un modelo de clasificación utilizado Aprendizaje de Máquina, y permitirá la elección de distintos algoritmos de aprendizaje. Las actividades a llevar a cabo en esta etapa son:

- Estudio de diferentes clasificadores provistos por WEKA.
- Estudio de la herramienta de procesamiento de lenguaje natural Google Natural Language.
- Diseño e implementación de la solución.
- Testeo y presentación de la solución para su uso como API REST,

Durante la etapa de análisis de los datos, teniendo como base la tendencia histórica de Bitcoin, se analizará cada intervalo de tiempo con el fin de mejorar la precisión sobre la tendencia generada por la herramienta. Estos resultados se expondrán por pantalla y se guardarán en archivos. A partir de los datos analizados también se podrán predecir tendencias de los intervalos clasificados. Para esto vamos a contar con las siguientes actividades:

- Análisis de los intervalos de tiempo.
- Definición de la entrada de la herramienta (etiquetas de tendencias sobre cada intervalo).
- Definición de tendencias como salida.

2. Marco Teórico

En este capítulo se describen los conceptos teóricos más relevantes necesarios para comprender el trabajo realizado, para orientar al lector en el dominio en el que se encuentra el mismo, es decir las tendencias del bitcoin y la Minería de Datos. A su vez, se presentan trabajos relacionados a ese dominio, realizando comparaciones y analizando puntos en común.

2.1 Qué es el Bitcoin y cómo funciona

El Bitcoin es una moneda, como el euro o el dólar, que sirve para intercambiar bienes y servicios. Sin embargo, a diferencia de las monedas mencionadas, el Bitcoin es una divisa electrónica, que existe exclusivamente en Internet, y que gana prestigio en el ciberespacio gracias a su eficiencia, seguridad y facilidad de intercambio.

Su principal diferencia frente al resto de monedas es que se trata de una moneda descentralizada, cuyo valor no depende de la tasa de interés que fija un Banco Central de Reserva ni tampoco de sus reservas en bancos comerciales. El Bitcoin se somete a las leyes de la oferta y la demanda, elevando o disminuyendo su valor sobre la base de Bitcoins disponibles o no en el mercado. Se sabe que se emitirán hasta 21 millones de Bitcoins al año 2040.

Se trata de la primera implementación de un concepto conocido como criptomoneda. La primera especificación del denominado protocolo Bitcoin se publicó en 2009 bajo el alias de Satoshi Nakamoto en una lista de correo electrónico. Este personaje digital abandonó el proyecto Bitcoin a finales de 2010 pero desde entonces existe una comunidad entusiasta que ha crecido de forma exponencial, gracias a la llegada de cada vez más desarrolladores que trabajan en el protocolo Bitcoin.

A continuación, se detallan cinco de sus propiedades transaccionales:

1. Irreversible: Después de la confirmación, una transacción no puede ser revertida.
2. Pseudónimos: Ni las transacciones ni las cuentas están conectadas a identidades del mundo real. Recibes Bitcoins en las denominadas “direcciones”, que son cadenas aparentemente aleatorias de alrededor de 30 caracteres. Si bien usualmente es posible analizar el flujo de transacciones, no es necesariamente posible conectar la identidad del mundo real de los usuarios de esas direcciones.

3. Rápido y global: Las transacciones son propagadas casi instantáneamente en la red y son confirmadas en unos pocos minutos. Dado que ocurren en una red global de computadoras son completamente indiferentes de la ubicación física. No importa si envías Bitcoins a un vecino o a alguien en el otro lado del mundo.
4. Seguro: Los fondos de Bitcoin están bloqueados en un sistema de criptografía de clave pública. Solo el propietario de la clave privada puede enviar criptomonedas. Una criptografía fuerte y la magia de números grandes hace imposible romper este esquema.
5. Sin permisos: No necesitas preguntar a nadie para usar criptomonedas. Es solo un software que cualquiera puede descargar de manera gratuita. Después de instalarlo, puedes recibir y enviar Bitcoins u otras criptomonedas. Nadie te lo puede impedir. No existe un guardián.

Bitcoin utiliza criptografía de clave pública en un enfoque innovador a la contabilidad para lograr la autorización, verificación de saldo, prohibición del doble gasto, entrega de activos e inalterabilidad de los registros mencionada anteriormente. Y ocurre casi en tiempo real y sin costos.

La criptografía asegura la autorización. Necesitas una clave privada para realizar una transacción. Y si la clave es lo suficientemente compleja que tomaría a la mejor computadora del mundo más tiempo de lo que existió la Tierra corromperla. En otras palabras, es esencialmente inmune a ataques.

2.2 Minería de datos

La revolución digital hizo posible que la información digitalizada sea fácil de capturar, procesar, almacenar, distribuir, y transmitir. Con el importante progreso en informática y en las tecnologías relacionadas y la expansión de su uso en diferentes aspectos de la vida, se continúa recogiendo y almacenando gran cantidad de información.

La minería de datos nació con la idea de aprovechar dos cosas: la gran cantidad de datos que se almacena, y la potencia de las nuevas computadoras para realizar operaciones de análisis sobre esos datos. El data mining permite encontrar información escondida en los datos que no

siempre resulta aparente, ya que, dado el gigantesco volumen de datos existentes, gran parte de ese volumen nunca será analizado. La minería de datos es un proceso de identificación de información relevante extraída de grandes volúmenes de datos, con el objetivo de descubrir patrones y tendencias estructurando la información obtenida de un modo comprensible para su posterior utilización (Agrawal et al., 1996).

La minería de datos surge a principios de los años ochenta cuando la Administración de Hacienda de Estados Unidos desarrolló un programa de investigación para detectar fraudes en la declaración y evasión de impuestos, mediante lógica difusa, redes neuronales y técnicas de reconocimiento de patrones. Sin embargo, su expansión se produce hasta la década de los noventa, principalmente debido a:

- El incremento en la potencia de procesamiento de las computadoras, así como en la capacidad de almacenamiento.
- El crecimiento de la cantidad de datos almacenados, el cual se ve favorecido no solo por el abaratamiento de los discos y sistemas de almacenamiento masivo, sino también por la automatización de trabajos y técnicas de acopio de datos (observación con nuevas tecnologías, entrevistas más prácticas, encuestas, chats, etc).
- La aparición de nuevos métodos y técnicas de aprendizaje y almacenamiento de datos, como las redes neuronales, la Inteligencia Artificial y el surgimiento del almacén de datos (Vallejo Ballesteros et al., 2018).

Un proceso típico de minería de datos consta de los pasos generales presentados en la Figura 2.1:

El Proceso de la Minería de Datos

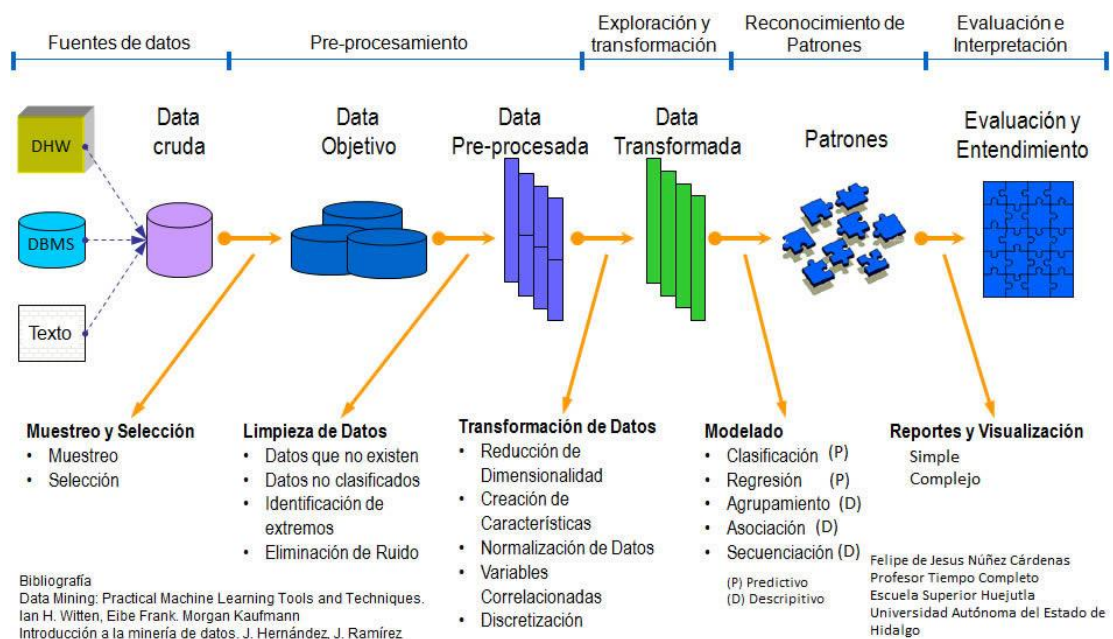


Figura 2.1. Proceso de minería de datos (Witten et al., 1999).

1. Selección del conjunto de datos, tanto en lo que se refiere a las variables objetivo (aquellas que se quiere predecir, calcular o inferir), como a las variables independientes (las que sirven para hacer el cálculo o proceso), como posiblemente al muestreo de los registros disponibles.
2. Análisis de las propiedades de los datos, en especial los histogramas, diagramas de dispersión, presencia de valores atípicos y ausencia de datos (valores nulos).
3. Transformación del conjunto de datos de entrada, se realizará de diversas formas en función del análisis previo, con el objetivo de prepararlo para aplicar la técnica de minería de datos que mejor se adapte a los datos y al problema, a este paso también se le conoce como preprocesamiento de los datos.
4. Seleccionar y aplicar la técnica de minería de datos, se construye el modelo predictivo, de clasificación o segmentación.
5. Extracción de conocimiento, mediante una técnica de minería de datos, se obtiene un modelo de conocimiento, que representa patrones de comportamiento observados en los valores de las variables del problema o relaciones de asociación entre dichas variables. También pueden usarse varias técnicas a la vez para generar distintos modelos, aunque generalmente cada técnica obliga a un preprocesado diferente de los datos.
6. Interpretación y evaluación de datos, una vez obtenido el modelo, se debe proceder a su validación comprobando que las conclusiones que arroja son válidas y suficientemente satisfactorias. En el caso de haber obtenido varios modelos mediante el uso de distintas técnicas, se deben comparar los modelos en busca de aquel que se ajuste mejor al problema. Si ninguno de los modelos alcanza los resultados esperados, debe alterarse alguno de los pasos anteriores para generar nuevos modelos.

Si el modelo final no superará esta evaluación el proceso se podría repetir desde el principio o, si el experto lo considera oportuno, a partir de cualquiera de los pasos anteriores. Esta retroalimentación se podrá repetir cuantas veces se considere necesario hasta obtener un modelo válido.

La minería de datos tiene como objetivo analizar los datos desde todas las perspectivas estratégicas. En el presente trabajo se va a extraer información de Twitter, con el fin de transformarla en información útil y conocimiento, y poder inferir las tendencias del Bitcoin, siguiendo los pasos del proceso típico de este campo, concluyendo con una evaluación final de los resultados.

Para finalizar, en la tabla 2.1 se muestra una lista de las principales técnicas de minería de datos (Weiss et al., 1998)(Molina López et al., 2006), desarrollando Redes Bayesianas, ya que luego va a ser utilizada en la herramienta.

Técnicas de Minería de Datos			
Técnicas	No supervisadas	Agrupamiento	Numérico
			Conceptual

			Probabilístico
		Asociación	A priori
		Predicción	Regresión
			Árboles de predicción
			Estimador de núcleos
	Supervisadas	Clasificación	Tabla de decisión
			Árboles de decisión
			Inducción de reglas
			Bayesianas
			Basada en ejemplos
			Redes neuronales

Tabla 2.1. Ejemplos de técnicas de minería de datos

2.2.1 Redes Bayesianas

Las redes bayesianas son grafos dirigidos acíclicos cuyos nodos representan variables aleatorias en el sentido de Bayes: las mismas pueden ser cantidades observables, variables latentes, parámetros desconocidos o hipótesis. Las aristas representan dependencias condicionales; los nodos que no se encuentran conectados representan variables las cuales son condicionalmente independientes de las otras. Cada nodo tiene asociado una función de probabilidad que toma como entrada un conjunto particular de valores de las variables padres del nodo y devuelve la probabilidad de la variable representada por el nodo (Luis M. de Campos et al., 2004).

En el caso de este trabajo, se puede realizar la aplicación de la Regla de Bayes para predecir la probabilidad condicional de que una tendencia pertenezca a una clase $P(c_i | r_j)$ a partir de la probabilidad a priori de la clase en el conjunto de entrenamiento $P(c_i)$.

$$P(c_i | r_j) = \frac{P(c_i) P(r_j | c_i)}{P(r_j)}$$

Dado que la probabilidad de cada tendencia $P(d_j)$ no aporta información para la clasificación, el término suele omitirse. La probabilidad de una tendencia dada la clase suele asumirse como la probabilidad conjunta de los features analizados dada la clase y se calculan como:

$$P(r_j | c_i) = \prod_{t=1}^{|v|} P(f_t | c_i)$$

Adicionalmente, el modelo Naive Bayes Multinomial considera la frecuencia de aparición de cada feature en las tendencias x_t en vez de una ocurrencia binaria:

$$P(r_j | c_i) = \prod_{t=1}^{|v|} P(f_t | c_i)^{x_t}$$

El término $P(f_t | c_i)$ se calcula a partir del número de apariciones de cada feature f_t en una clase c_i pero para evitar el problema de las probabilidades 0 se usa la estimación de Laplace:

$$P(f_t | c_i) = \frac{1 + n(f_t | c_i)}{|v| + n(c_i)}$$

Donde $n(f_t | c_i)$ es el número de ocurrencias de f_t en c_i , $|v|$ es el tamaño del dataset y $n(c_i)$ es el conteo total del feature en c_i . De este modo, la clasificación se realiza buscando la tendencia que maximiza la función:

$$c^*(d) = \underset{c_i}{\text{tendenciamax}} P(c_i) \prod_{t=1}^{|v|} P(f_t | c_i)^{x_t}$$

2.2.2 Algoritmos utilizados

Decision Table

Una tabla de decisiones es una entrada de lógica de reglas planificadas, en formato de tabla, que se compone de condiciones, representadas en las cabeceras de columna y fila, y acciones, representadas como puntos de intersección de los casos condicionales de la tabla. Las tablas de decisiones son especialmente idóneas para las reglas de negocio que tienen varias condiciones. Añadir otra condición es tan fácil como añadir otra fila o columna.

Al igual que el conjunto de reglas if/then (si/entonces), la tabla de decisiones es controlada por la interacción de condiciones y acciones. La diferencia principal radica en que en una tabla de decisiones, la acción se decide a través de más de una condición, y se puede

asociar más de una acción con cada conjunto condiciones. Si se cumplen las condiciones, se realiza la acción o acciones correspondientes.

Naive Bayes

En términos simples, un clasificador de Naive Bayes asume que la presencia o ausencia de una característica particular no está relacionada con la presencia o ausencia de cualquier otra característica, dada la clase variable. Por ejemplo, una fruta puede ser considerada como una manzana si es roja, redonda y de alrededor de 7 cm de diámetro. Un clasificador de Naive Bayes considera que cada una de estas características contribuye de manera independiente a la probabilidad de que esta fruta sea una manzana, independientemente de la presencia o ausencia de las otras características.

Para otros modelos de probabilidad, los clasificadores de Naive Bayes se pueden entrenar de manera muy eficiente en un entorno de aprendizaje supervisado. En muchas aplicaciones prácticas, la estimación de parámetros para los modelos Naive Bayes utiliza el método de máxima verosimilitud, en otras palabras, se puede trabajar con el modelo de Naive Bayes sin aceptar probabilidad bayesiana o cualquiera de los métodos bayesianos.

Una ventaja del clasificador de Naive Bayes es que solo se requiere una pequeña cantidad de datos de entrenamiento para estimar los parámetros (las medias y las varianzas de las variables) necesarias para la clasificación. Como las variables independientes se asumen, solo es necesario determinar las varianzas de las variables de cada clase y no toda la matriz de covarianza.

SMO

La optimización mínima secuencial (SMO) es un algoritmo para resolver el problema de la programación cuadrática (QP) que surge durante el entrenamiento de las máquinas de vectores de soporte (SVM).

SMO es ampliamente utilizado para la capacitación de máquinas de vectores de soporte. La publicación del algoritmo SMO en 1998 ha generado mucha emoción en la comunidad SVM, ya que los métodos disponibles anteriormente para el entrenamiento SVM eran mucho más complejos y requerían costosos solucionadores de QP de terceros.

J48

J48 es una implementación open source en lenguaje de programación Java del algoritmo C4.5 en la herramienta weka de minería de datos.

C4.5 es un algoritmo usado para generar un árbol de decisión desarrollado por Ross Quinlan.¹ C4.5 es una extensión del algoritmo ID3 desarrollado anteriormente por Quinlan.

Los árboles de decisión generados por C4.5 pueden ser usados para clasificación, y por esta razón, C4.5 está casi siempre referido como un clasificador estadístico.

SGD

Stochastic Gradient Descent, es un clasificador en el cual algunas muestras se seleccionan aleatoriamente en lugar del conjunto de datos completo para cada iteración. En Gradient Descent, hay un término llamado "lote" que denota el número total de muestras de un conjunto de datos que se utiliza para calcular el gradiente para cada iteración. En la optimización típica de Gradient Descent, como Batch Gradient Descent, el lote se considera el conjunto de datos completo. Aunque, usar todo el conjunto de datos es realmente útil para llegar a los mínimos de una manera menos ruidosa y menos aleatoria, pero el problema surge cuando nuestros conjuntos de datos aumentan.

2.3 Aprendizaje de máquina

La minería de datos utiliza en su análisis los métodos del aprendizaje de máquina. Este (del inglés, "Machine Learning") es un subcampo de las ciencias de la computación y una rama de la inteligencia artificial, cuyo objetivo es desarrollar técnicas que permitan que las computadoras aprendan (Jeria,. 2007). De forma más concreta, se trata de crear algoritmos capaces de generalizar comportamientos y reconocer patrones a partir de una información suministrada en forma de ejemplos. Es, por lo tanto, un proceso de inducción del conocimiento, es decir, un método que permite obtener por generalización un enunciado general a partir de enunciados que describen casos particulares.

En muchas ocasiones el aprendizaje de máquina se solapa con la Minería de Datos, ya que las dos disciplinas están enfocadas en el análisis de datos. Sin embargo, el aprendizaje de máquina se centra más en el estudio de la complejidad computacional de los problemas desde el punto de vista práctico, no únicamente teórico.

A un nivel muy básico, se puede decir que una de las tareas del aprendizaje de máquina es intentar extraer conocimiento sobre algunas propiedades no observadas de un objeto basándose en las propiedades que sí han sido observadas, en pocas palabras, predecir comportamiento futuro a partir de lo que ha ocurrido en el pasado.

Hay un gran número de problemáticas que caen dentro de lo que llamamos aprendizaje de máquina. La principal diferencia entre ellos radica en el tipo de objetos que intentan predecir. Algunas clases habituales son regresión, clasificación, ranking, etc.

El aprendizaje de máquina tiene como resultado un modelo para resolver una tarea dada. Entre los modelos se distinguen

- Los modelos geométricos, contruidos en el espacio de instancias y que pueden tener una, dos o múltiples dimensiones. Si hay un borde de decisión lineal entre las clases, se dice que los datos son linealmente separables. Un límite de decisión lineal se define como $w * x = t$, donde w es un vector perpendicular al límite de decisión, x es un punto arbitrario en el límite de decisión y t es el umbral de la decisión.
- Los modelos probabilísticos, que intentan determinar la distribución de probabilidades descriptora de la función que enlaza a los valores de las características con valores determinados. Uno de los conceptos claves para desarrollar modelos probabilísticos es la estadística bayesiana.
- Los modelos lógicos, que transforman y expresan las probabilidades en reglas organizadas en forma de árboles de decisión.

Este aprendizaje, además, puede clasificarse en Aprendizaje supervisado, no supervisado, o semi supervisado (Dubiau, 2013), dependiendo del tipo de salida que se produzca y de cómo se aborde el tratamiento de los ejemplos.

- Aprendizaje supervisado: se genera una función que establece una correspondencia entre las entradas y las salidas deseadas del sistema, donde la base de conocimientos está formada por ejemplos etiquetados a priori. Es decir, ejemplos de los que se conoce su clasificación correcta.
- Aprendizaje no supervisado: donde el proceso de modelado se lleva a cabo sobre un conjunto de ejemplos formados únicamente por entradas al sistema, sin conocer su clasificación correcta. Se busca que el sistema sea capaz de reconocer patrones para poder etiquetar las nuevas entradas.
- Aprendizaje semi supervisado: Este tipo de algoritmos combinan los dos algoritmos anteriores para poder clasificar de manera adecuada. Se tiene en cuenta los datos marcados y los no marcados.

2.4 Procesamiento de Lenguaje Natural

Una de las ramas más importantes de la Inteligencia Artificial es aquella orientada a facilitar la comunicación hombre-computadora por medio del lenguaje humano, o lenguaje natural. El Procesamiento del Lenguaje Natural (PLN) es la disciplina encargada de producir sistemas informáticos que posibiliten dicha comunicación. Las aplicaciones de PLN necesitan estudiar el lenguaje natural en profundidad y para esto utilizan distintos tipos de análisis (Cortez et al., 2014).

El estudio del lenguaje natural se estructura normalmente en los siguientes niveles:

- **Análisis Morfológico:** la morfología es la rama de la lingüística que se preocupa por la descripción de la estructura de las palabras y el proceso de formación de las mismas. La idea general es que los morfemas individuales pueden ser combinados para formar palabras.
- **Sintáctico:** la sintaxis, o construcción de oraciones, es el nivel más bajo en el cual el lenguaje humano es constantemente creativo. El conocimiento sintáctico es un componente básico de cualquier sistema de PLN, ya que se encarga de reconocer las oraciones gramaticales y asignarles una estructura. El reconocimiento de la estructura de las oraciones por parte de la computadora es llevado a cabo por un algoritmo llamado parsing.
- **Semántico:** la semántica, o significado, es el nivel en el cual el lenguaje hace contacto con el mundo real. Se trata de la primera tarea del componente interpretativo, la cual consiste en asignar un significado a cada una de las oraciones analizadas independientemente del contexto. La semántica oracional es una parte imprescindible de cualquier sistema, ya que sin ella no podríamos asignar significado a las estructuras analizadas.
- **Pragmático:** se refiere al uso del lenguaje en el contexto. En general la pragmática incluye aspectos del conocimiento conceptual del mundo que van más allá de las condiciones reales literales de cada oración. Este conocimiento lo tienen en cuenta los hablantes cuando se comunican mediante una lengua. Les sirve para comprender mucha información sobreentendida pero no expresada explícitamente en las oraciones. Mientras la sintaxis y semántica estudia las oraciones, la pragmática estudia “las acciones del discurso” y las situaciones en las cuales el lenguaje es usado.

Cuando se analiza texto, el resultado del análisis generalmente se representa mediante anotaciones o etiquetas. Una operación de etiquetado o anotación se basa en un conjunto de etiquetas que representan las diversas categorías lingüísticas o extra-lingüísticas.

Las anotaciones pueden tener atributos, por ejemplo, un análisis que busca todos los nombres propios de personas en un texto (María, Pedro, etc.), podría etiquetar o anotar cada nombre y además, a cada anotación agregar un atributo que diga si el nombre es femenino o masculino. El hecho que las anotaciones tengan atributos permite representar otros tipos de análisis más complejos ya que una anotación puede tener a otras anotaciones como atributos.

2.5 Tendencias en Twitter

Al igual que muchas otras redes sociales, Twitter es una gran fuente de datos. Por ello, gran multitud de estudios enmarcados en el campo de la minería de datos se han centrado en esta red social.

Sin duda, estudios basados en marketing destacan sobre muchos otros trabajos y proyectos desarrollados en torno a Twitter. Las marcas aprovechan toda la información que los

usuarios generan al comunicarse con familiares y amigos, explotándola y descubriendo sus intereses, además de conocer qué se está comentando sobre una marca en concreto. También envían spam a los usuarios de Twitter, y este estudio recoge que esta red social es muy susceptible a ese tipo de prácticas. Se analizan marcas de distintos sectores: marcas del sector textil, de la industria automovilística, del sector de hotelero, etc.

Han aparecido varios trabajos que muestran el potencial de Twitter para poder hacer seguimiento y predicción de brotes de enfermedades. Los usuarios de Twitter pueden publicar comentarios acerca de enfermedades y sus síntomas, y realizando una geolocalización de los mismos se podrían utilizar para detectar zonas de posibles brotes de enfermedades.

Basándose en esta idea, la universidad de Northwestern, por ejemplo, ha desarrollado un sistema para detectar brotes de gripe (Lee K. et al., 2013). El sistema extrae continuamente textos relacionados con enfermedades y utiliza técnicas de minería de datos basados en texto para mostrar gráficas y mapas de la enfermedad.

Por último, también se han llevado a cabo trabajos que estudian el impacto de Twitter en otros medios de comunicación, en concreto, la televisión (Highfield T. et al., 2013).

2.7 Trabajos relacionados

Existen trabajos pertenecientes al dominio de estudio de las tendencias teniendo como fuente los comentarios de Twitter que alentaron al actual trabajo.

Se observaron propuestas en donde se realiza análisis de sentimiento para predecir la tendencia del Bitcoin en redes sociales. En el trabajo presentado por Herrero Amorós et al., (2018), se realiza un análisis acerca de la predicción de la evolución de la cotización del Bitcoin. Se describe el proceso de obtención de datos mediante técnicas de scraping sobre diferentes fuentes de datos utilizadas: Binance, Twitter y Google Trends. Se han tenido en cuenta diferentes índices de mercado que rigen la criptomoneda, como son el volumen y precios de apertura o cierre, entre otros. Además, se han tratado índices de polaridad de sentimientos y popularidad a partir de Twitter y Google Trends respectivamente. A partir de los datos obtenidos, se han aplicado procesos de feature engineering y minería de textos que han permitido la obtención de información implícita sobre la información original que complementa y enriquece el conjunto de datos. El modelado de los datos se ha realizado utilizando técnicas estadísticas, como son redes neuronales del tipo Long-Short Term Neural Network u otras como por ejemplo Gradient Boosting, el cual demostró tener mejor precisión, obteniendo un accuracy del 96% en su configuración óptima.

En estos contextos, se observó la falta de una herramienta que automatice la labor de las personas encargadas de la clasificación de las tendencias, por lo que se propone este trabajo para acelerar la caracterización de las tendencias, disminuyendo el alto consumo de recursos humano-temporal que requiere el proceso de supervisión y categorización de interacciones por parte de personas idóneas en el tema. El enfoque es similar al que se propone, buscando una

correlación entre la cantidad de tweets con determinando sentimiento (positivo-negativo) y la tendencia.

El trabajo mencionado, alienta el potencial de trabajar con análisis de sentimientos y de esta forma predecir diferentes tendencias según el campo de interés.

También en el trabajo descrito en (McAteer Ciaran, 2014), se busca una correlación entre el precio del Bitcoin y el sentimiento de los Tweets en los que hablan sobre esta criptomoneda. En este trabajo se utiliza una herramienta llamada Mahout Project, la cual es un proyecto de Apache Software Foundation para producir implementaciones gratuitas de algoritmos de Aprendizaje de Máquina. Esta herramienta se utilizó para clasificar los tweets según su sentimiento mediante diferentes algoritmos de aprendizaje obteniendo un accuracy del 80.28%.

De esta forma, se llegó a la conclusión de que los inversores son propensos al sentimiento de las noticias ya que las noticias malas o buenas a menudo se difunden en Twitter, como por el ejemplo en un retweet con sentimiento negativo como “RT @ BitcoinAgile: el precio de Bitcoin cae bruscamente a raíz del informe del gobierno de EE. UU.” obtuvo cientos de retweets, incrementando el número de tweets con sentimiento negativo ante una fuerte caída en el precio del mismo. Además, también se llegó a la conclusión de que el número de tweets relacionados con el Bitcoin, está correlacionado con la fluctuación de precios del mismo.

Este trabajo a diferencia con del trabajo que se propone, se vuelve a observar la falta de automatización en el proceso ya que la clasificación se realiza a través de una herramienta genérica, implicando mayor esfuerzo manual al no contar con un software a medida que realice el procedimiento.

Esto mismo ocurre con el aporte presentado por Georgoula, Ifigeneia, et al., (2015) ya que no cuenta con un software que facilite el proceso general de los tweets. Por otro lado, en este trabajo se capturaron 2,125,243 tweets durante un periodo de 78 días y se obtuvo un accuracy del 89%. Se llegó a la conclusión que la evidencia muestra que las mediciones del estado de ánimo colectivo se basan en el análisis de sentimiento apropiado puede ayudar a predecir movimientos a corto plazo en el valor de Bitcoin.

3. Desarrollo de la propuesta

En este capítulo se desarrolla la propuesta que motiva al presente trabajo, detallando las principales decisiones de diseño que llevaron a la implementación del mismo. Se presentará la estructura de la herramienta, junto con detalles a tener en cuenta sobre la misma, y el enfoque propuesto para inferir la tendencia del Bitcoin.

3.1. Estructura de la herramienta desarrollada

Mediante las tecnologías mencionadas en el Capítulo 2, se creó una plataforma capaz de predecir la tendencia del Bitcoin según las publicaciones que realizan los usuarios en Twitter.

La herramienta desarrollada es una aplicación de escritorio, la cual a partir una entrada determinada es capaz de inferir la tendencia (a la suba o a la baja) en el valor del Bitcoin, utilizando modelos de aprendizaje y análisis del lenguaje natural. Se dividirá a la misma en dos partes, el entrenamiento y la predicción, siguiendo el proceso de minería de datos.

El dataset, como se mencionó anteriormente, proviene de la captura de tweets durante el periodo de tiempo comprendido entre el [30/11/2017 - 14/02/2018]. Para llevar a cabo esta captura se dejó en funcionamiento una aplicación implementada en java utilizando el framework Twitter4j. Twitter4J es una librería Java para manejar la API de Twitter desde Java. Es una librería no oficial aunque por su sencillez, potencia y capacidades es probablemente la más usada. Dicho framework permite implementar un listener capaz de capturar los tweets en tiempo real con formato Json para luego ser almacenados en un dataset.

A estos tweets se le aplicará procesamiento de lenguaje natural utilizando GNL para darle un valor sentimental al texto que contiene un conjunto de tweets. Se analizaron otras opciones de herramientas que realizan análisis de sentimiento, como Stanford NLP y Apache OpenNLP, pero GNL resultó ser la herramienta con mayor documentación y soporte. Además resultó ser la que mejor acertaba el sentimiento de los tweets ya que los mismos no están escritos generalmente en oraciones bien formadas sintácticamente y la herramienta provee con

un gran soporte al momento de analizar este tipo de textos. Una limitante de esta herramienta es el tiempo de respuesta (alrededor de un segundo por cada petición), la cual torna imposible el seguimiento en tiempo real teniendo en cuenta la gran magnitud de tweets que circula por la red social. Por tal motivo, solo se utilizó GNL para la etapa de entrenamiento, en donde se etiquetó un conjunto de oraciones extraídas de tweets que contengan la palabra “bitcoin”.

En este caso, la variable objetivo (es decir, la variable que se quiere predecir) es si el sentimiento expresado en texto es positivo, negativo o neutral.

A este conjunto preprocesado de datos se aplicarán diferentes técnicas de aprendizaje de máquina, a partir de algoritmos para análisis de datos y modelado predictivo provistos por la herramienta Weka, como Bayes (Naive Bayes y Bayes Net), árboles de decisión(J48), máquinas de vectores de soporte (SMO), basados en reglas (Part, JRip, Decision Table) y meta clasificadores (Logit Boost, Multiclass Classifier). Al utilizar la librería de aprendizaje de máquina WEKA, los datos de entrada a la herramienta sobre los que operan las técnicas implementadas, deben estar codificados en un formato específico, denominado Attribute-Relation File Format (extensión "arff").

Una vez aplicadas las distintas técnicas, se procederá al análisis de los datos obtenidos como salida. Buscando de esta forma, obtener un conjunto de métricas que nos permita encontrar una relación entre la tendencia del Bitcoin y los comentarios sobre esta criptomoneda que circulan en Twitter.

La herramienta consta de las siguientes funcionalidades:

- Convertir archivo en formato JSON a arff
- Crear clasificador a partir de un archivo arff
- Clasificar archivos en formato Json (utilizando el clasificador previamente cargado con modelos de prueba)

Convertir archivo Json a arff

Como fuente de entrada se tiene un archivo en formato JSON proveniente de Twitter, siendo un tweet con su respectiva metadata, en la Tabla 3.1 puede verse un ejemplo reducido.

```
{  
  "created_at": "Thu Apr 06 15:24:15 +0000 2017",
```

```

    "id_str": "850006245121695744",
    "text": "1\ Today we\u2019re sharing our vision for the
future
of
the
Twitter
API
platform!\nhttps:\\\\t.co\\XweGngmxlP",
    "user": {
      "id": 2244994945,
      "name": "Twitter Dev",
      "screen_name": "TwitterDev",
      "location": "Internet",
      "url": "https:\\\\dev.twitter.com\\",
      "description": "Your official source for Twitter Platform
news, updates & events. Need technical help? Visit
https:\\\\twittercommunity.com\\
#TapIntoTwitter"
    },
    "place": {
    },
    "entities": {
      "hashtags": [
      ],
      "urls": [
        {
          "url": "https:\\\\t.co\\XweGngmxlP",
          "unwound": {
            "url":
"https:\\\\cards.twitter.com\\cards\\18ce53wgo4h\\3xo1c",
            "title": "Building the Future of the Twitter API
Platform"
          }
        }
      ]
    },
    "user_mentions": [
    ]
  }
}

```

Tabla 3.1. Ejemplo de tweet en formato JSON

Como puede observarse, Twitter ofrece una serie de campos útiles que enriquecen la información del tweet y del mismo se extraerán un conjunto de valores seleccionados para luego convertirlos en arff.

Crear clasificador a partir de un archivo arff

Dado un archivo en formato arff, se debe elegir el atributo a clasificar (ClassIndex) para poder así construir el modelo siguiendo el enfoque propuesto.

Clasificar archivos en formato JSON

La herramienta permitirá clasificar un archivo en formato JSON conteniendo los tweets de la ventana de tiempo que se desea analizar. Devolviendo la cantidad de negativos, positivos y neutrales.

3.2. Enfoque de clasificación de propuesto

En esta sección se presentará el enfoque propuesto para clasificar el sentimiento de los tweets, como el entrenamiento del mismo. Este enfoque recibe el nombre de “Ranker”.

3.2.1 Enfoque propuesto

El enfoque Ranker propone particionar tendencias en ventanas de tiempo, para el caso de estudio se proponen ventanas de veinticuatro horas.

Como hipótesis se busca una relación entre la tendencia del Bitcoin y la cantidad de Tweets con sentimiento positivo y negativo durante esa ventana de tiempo.

Es decir, si durante la ventana de tiempo tiene mayor cantidad de tweets positivos, será clasificado como tendencia alcista. Análogamente en caso de ser predominante la cantidad de tweets con sentimientos negativos.... será clasificado como tendencia a la baja.

La estrategia de este enfoque propone ignorar la metadata y considerar solo la cantidad palabras de interés, las cuales son seleccionadas por el usuario.

3.2.2 Pasos previos

Cabe destacar que para poder llegar a aplicar el enfoque, se completaron una serie de pasos previos a la aplicación del enfoque.

Durante la etapa de entrenamiento se capturaron Tweets durante un periodo de tres meses las veinticuatro horas de cada día. Dicha captura fue llevada a cabo mediante una instancia de Amazon Elastic Compute Cloud, conocida como EC2 (detallado en 8.2) de AWS corriendo un jar haciendo función de consumidor de Twitter utilizando la librería Twitter4 y almacenando los tweets en formato JSON. La ventaja de utilizar es EC2 es la disponibilidad que garantiza sumado a su espacio de almacenamiento.

Se desarrolló un listener realizando la función de capturar los tweets. Dicho listener provee un método muy útil (*onStatus*) para llevar a cabo el trabajo ya que al hacerlo a medida, se podrá tomar acción sobre la captura de cada tweet y filtrar los tweets por palabras clave, lenguaje en el que está escrito, la zona geográfica, etc.

Al proceder a almacenar en formato JSON, se generó demasiada información sobre los tweets ya que cada uno tiene más de cien atributos y se decidió solo almacenar los siguientes ya que a priori parecieron las más relevantes:

- lang: Lenguaje en que se encuentra el tweet,
- created_at: Fecha de creación del tweet,
- isRetweet: Indica si la publicación es un retweet.
- verified: Indica si la cuenta está verificada
- followers_count: Cantidad de seguidores que tiene el usuario creador del tweet.
- listed_count: El número de listas públicas de las que es miembro este usuario.,
- statuses_count: Cantidad de publicaciones que realizó el usuario creador del tweet.
- name: Nombre del usuario.
- description: Descripción del usuario.
- friends_count: Cantidad de amigos que tiene el usuario en Twitter.
- location: Indica si la publicación tiene información sobre la ubicación.
- hashtags: Los hashtags que tiene la publicación.

Las principales razones por la que se decidió tomar la decisión de almacenar solo estos campos son, el gran tamaño de almacenamiento que se requiere para almacenar todos y el tiempo de procesamiento de los mismos. Por tal motivo, además se optó por seleccionar solo los tweets que contienen la palabra clave "*bitcoin*".

Otros motivos ajenos al filtro de atributos que aparecen en este caso son la cantidad excesiva de tweets que circula por el mundo, el tamaño de almacenamiento necesario para guardar y el límite de tweets que permite capturar Twitter por periodo de tiempo.

Una vez obtenidos los tweets, se procede a analizar el sentimiento del tweet.

Al momento de analizar se tiene información sobre el texto (por quién fue provistas y demás datos que provee el JSON de cada Tweet).

Para etiquetar el texto equivalente al campo "text" del tweet con su valor de sentimiento correspondiente, se utilizó "Google Natural Language". Lo ideal sería utilizar constantemente GNL, pero por cuestiones de costos y latencia (ya que es un servicio en la nube), solo se utilizó para etiquetar veinte mil tweets y de esta forma saber a qué clase pertenece. Con este dataset, se generó un modelo capaz de catalogar el valor sentimental de nuevos tweets.

Cabe destacar que esta herramienta de Google es gratuita hasta las primeras 20.000 clasificaciones, por tal motivo ese es el número de registros con etiqueta. Si bien la herramienta puede tener como salida por cada registro valores (-1 ; 1), se decidió discretizar en tres grupos, negativos (-1 ; -0,33], neutrales (-0,33 ; 0,33] y positivos (0,33 ; 1).

Con los Tweets ya almacenados y con el valor sentimental provisto por GNL, se procede a procesarlos para generar los dataset en formato arff. Se eligió llevarlo a arff porque es el formato con el que trabaja la herramienta de Aprendizaje de Máquina “Weka”.

Una vez obtenido el dataset se procede a generar el modelo capaz de catalogar el valor sentimental del texto que genera el tweet.

4. Diseño e implementación de la herramienta propuesta

En esta sección se va a detallar el diseño e implementación para entrenar a los clasificadores que se utilizan para identificar las tendencias del Bitcoin. La misma describe las herramientas que se utilizaron, los distintos requisitos funcionales y atributos de calidad que se detectaron sobre el trabajo desarrollado, diferentes diagramas para plasmar las decisiones de diseño, y escenarios de calidad.

4.1 Herramientas utilizadas

4.1.1 Twitter4J

Twitter proporciona a las empresas, los desarrolladores y los usuarios, acceso a los datos de Twitter mediante su API

En un nivel superior, las API son la forma en que los programas informáticos "hablan" entre sí para solicitarse y enviarse información. Para esto, se le permite a una aplicación de software que llame a lo que se denomina punto de conexión: una dirección que corresponde a un tipo específico de información que proporciona (generalmente, los puntos de conexión son únicos, como los números telefónicos). Twitter permite acceder a partes de su servicio mediante las API para permitirles a las personas crear software que se integre con Twitter, como una solución que ayuda a una empresa a responder a la opinión del cliente en Twitter.

La API ofrece acceso amplio a los datos de Twitter que los usuarios han decidido compartir con el mundo, permitiendo a los usuarios administrar su propia información de Twitter que no es pública (como los Mensajes directos).

Para este trabajo se utilizó la librería “Twitter4J” para manejar la API de Twitter desde Java. Es una librería no oficial aunque por su sencillez, potencia y capacidades es probablemente la más usada.

4.1.2 Google Natural Language (GNL)

Google Natural Language revela la estructura y el significado del texto con modelos potentes de aprendizaje de máquina previamente entrenados en una API de REST fácil de usar y con modelos personalizados que se pueden crear con facilidad.

Se puede utilizar Natural Language para extraer información sobre personas, lugares, eventos y muchos otros datos, que se mencionan en documentos de texto, artículos periodísticos o entradas de blog. También puede ser utilizado para comprender las opiniones sobre tus productos expresadas en los medios sociales o analizar la intención en las conversaciones de los clientes que se den en un centro de atención telefónica o una aplicación de mensajería.

4.1.3 WEKA

Weka (Waikato Environment for Knowledge Analysis) es una plataforma de software para el aprendizaje de maquina y la minería de datos escrito en Java y desarrollado en la Universidad de Waikato. Weka es software libre distribuido bajo la licencia GNU-GPL.

El paquete Weka contiene una colección de herramientas de visualización y algoritmos para análisis de datos y modelado predictivo, unidos a una interfaz gráfica de usuario para acceder fácilmente a sus funcionalidades.

Weka soporta varias tareas estándar de minería de datos, especialmente, preprocesamiento de datos, clustering, clasificación, regresión, visualización, y selección. Todas las técnicas de Weka se fundamentan en la asunción de que los datos están disponibles en un fichero plano o una relación, en la que cada registro de datos está descrito por un número fijo de atributos (normalmente numéricos o nominales, aunque también se soportan otros tipos).

4.2. Requerimientos funcionales y restricciones de diseño

Al pensar esta aplicación, se espera que el usuario pueda obtener modelos finales para poder aplicarlos en la predicción de las tendencias de Bitcoin. Estos, a su vez, son creados cuando el usuario lo desee y no deben tardar demasiado tiempo en crearse, por lo tanto, la cantidad de features a la hora de seleccionar debe ser correctamente estudiada para que no tenga gran impacto en la performance. Cabe destacar que el aprendizaje no es incremental, ya que el modelo se crea a partir del dataset que se seleccione como entrada sin tener en cuenta lo anterior.

Analizando las distintas características que se esperan de la herramienta se obtuvieron los siguientes requerimientos funcionales

- La herramienta debe cumplir un propósito similar a un ETL (Extract, Transform and Load, en español Extraer, Transformar y Cargar).
- La herramienta debe ser capaz de crear múltiples modelos.
- La herramienta debe permitir la carga de archivos de entrenamiento.
- La herramienta debe permitir elegir entre distintos enfoques.
- La herramienta debe permitir la carga de tweets.

Por otra parte, se detectaron los siguientes requisitos no funcionales y atributos de calidad:

- Usabilidad: El sistema debe permitir al usuario entrenar a los clasificadores de manera sencilla.

- **Fiabilidad:** El sistema debe ser capaz de funcionar sin fallas dentro de las condiciones normales.
- **Precisión:** Se debe buscar la mayor precisión en el modelo final y tratar de minimizar el overfitting.
- **Modificabilidad:** El sistema debe permitir el ingreso de nueva meta información y adaptarla para incorporar al proceso de detección de tendencias fácilmente.

- **Performance:** La creación de los modelos finales no deben demorar mucho tiempo en su creación, ni necesitar mucho consumo de memoria
- **Portabilidad:** La aplicación de entrenamiento debe poder usarse en cualquier sistema operativo.

Especificación de requerimientos

A continuación se describen de forma general los requerimientos de la aplicación mediante el diagrama de Casos de Uso de la Figura 4.1 , los cuales serán detallados luego:

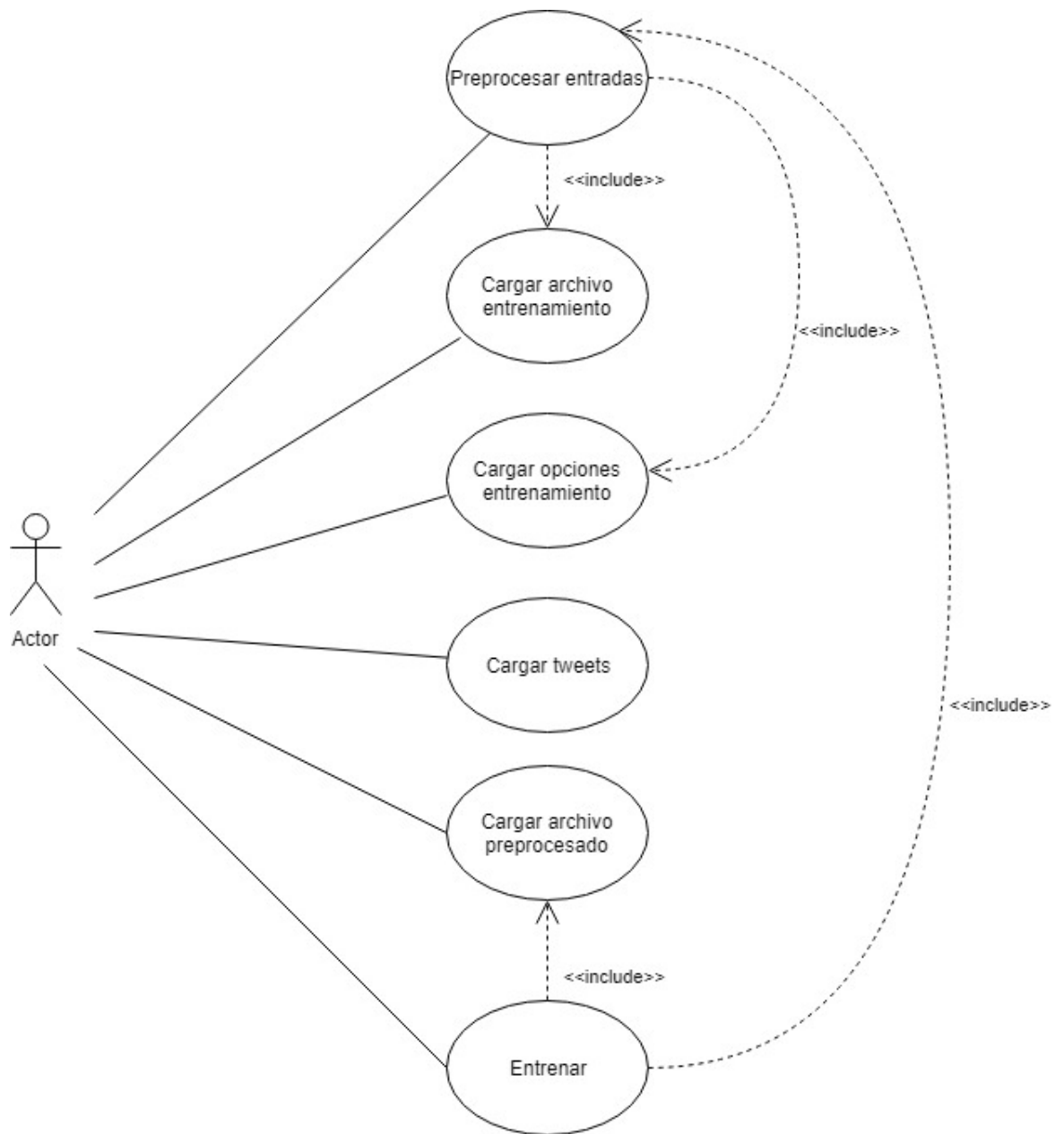


Figura 4.1. Diagrama de Casos de Uso

1. **Cargar archivo entrenamiento:** Se le debe permitir al usuario cargar el archivo que contenga los tweets con las que desee entrenar los modelos. No debe existir límite de tamaño, ya que los mismos suelen pesar mucho.
2. **Cargar opciones entrenamiento:** El usuario debe poder elegir el tipo de entrenamiento que desee. También debe poder permitir seleccionar el clasificador que desea entrenar.
3. **Cargar tweets:** El usuario debe contar con la opción de cargar los tweets que van a ser utilizadas para extraer el sentimiento de cada uno de ellos.
4. **Preprocesar entradas:** El usuario debe poder pre procesar un conjunto de entradas para utilizarlas en el entrenamiento, sin tener que demorar el procesamiento por cada clasificador que quiera entrenar.

5. **Cargar archivo preprocesado:** El usuario debe poder cargar un archivo ya procesado con los atributos esperados para realizar el entrenamiento de forma directa.
6. **Entrenar:** El usuario debe poder contar con la opción de entrenar al clasificador seleccionado. Una vez que termina el entrenamiento, el usuario debe poder observar los resultados.

4.3 Diseño de la solución

La herramienta puede ser observada como un ETL, donde en primera etapa se realiza una extracción de los datos, obteniendo así la información de los tweets, luego a esta se la preprocesa, se realizan diversos tipos de transformaciones, ya sea procesamiento de lenguaje natural y luego como último en la etapa de carga, que es en realidad es la creación del modelo final y no una carga a otra plataforma, donde esta ultima seria la diferencia con el proceso tradicional de ETL que se suele ver.

Para obtener una vista del diseño de la solución, se comienza desde el más alto nivel, por lo que la primera ilustración es sobre los componentes presentes en el desarrollo, representados en la Figura 4.2.

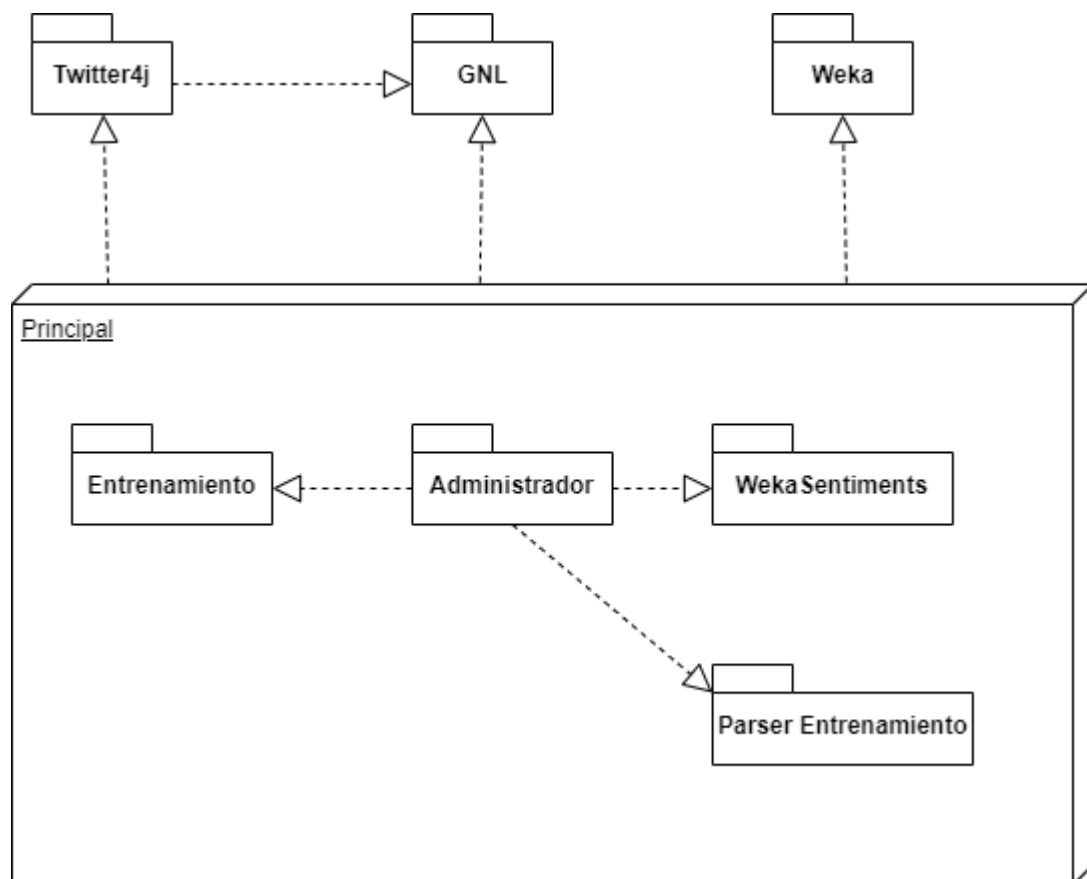


Figura 4.2. Diagrama de componentes

Dentro de los componentes principales, se encuentra el Administrador. Este no solo es aquel que se encarga de comunicar al resto de los componentes de realizar su tarea, sino que también es el encargado de la interfaz gráfica con el cual el usuario va a interactuar, por lo tanto, cada acción que el usuario realice va a impactar en alguno de los componentes restantes que será comunicado a través del mismo.

Luego se encuentran los entrenamientos, encargados de crear parte de los modelos, ya que dentro se encuentran los diferentes clasificadores que se entrenan y almacenan en sus respectivos archivos.

Antes de ser utilizados por estos entrenamientos, los conjuntos de datos pasan por los componentes de Parser. Entrenamiento encargados de mapear la información de forma correcta para que sea entendida por los siguientes procesos, tales como el componente de Organizador, que se encargan de resumir la información reduciendo registros y a su vez agrega más información al conjunto de características del modelo.

Por último se encuentra el componente WekaSentiments, el cual contiene procedimientos relacionados a Weka y los sentimientos de los tweets, profundizado más adelante.

A su vez fuera de los componentes principales, se encuentra Twitter4j utilizado para extraer los tweets, el cual a su vez utiliza GNL, herramienta que también es utilizada por los componentes principales y por último Weka que contiene todos los clasificadores, filtros y demás herramientas para el procesamiento de los datos y creación de efectivos modelos.

Una vez presentado el diseño de la solución, se va a detallar el cumplimiento de cada atributo de calidad a partir de distintos escenarios.

4.4 Escenarios de calidad

4.4.1 Usabilidad

Escenario de calidad 1. El usuario debe contar con una interfaz para poder ingresar archivos de entrenamiento y todo lo necesario para entrenar a los distintos clasificadores ofrecidos. Además debe poder visualizar los resultados del proceso.

- Fuente: Usuario.
- Estímulo: Entrenar a los clasificadores.
- Artefacto: Aplicación desarrollada.
- Ambiente: En tiempo de ejecución.
- Respuesta: El sistema debe contar con interfaz para el entrenamiento.
- Medida Rta: La interfaz no debe afectar a la performance del entrenamiento.

Para la carga de datos y el entrenamiento, se cuenta con la interfaz de usuario de la Figura

Mediante la misma, el usuario puede cargar el archivo principal de entrenamiento, junto con los archivos auxiliares de los tweets. Con estos datos ya cargados el usuario cuenta con la opción de pre procesarlos, así quedan listos para entrenar, sin necesidad de realizarlo cada vez que deseen entrenar a un clasificador.

La herramienta permite también cargar un archivo preprocesado, en el caso de que el usuario ya cuente con uno que cumpla con los requisitos del entrenamiento, seleccionar del clasificador que desea utilizar, y el tipo de procesamiento. También, se presenta en una vista los resultados obtenidos.

4.4.2 Fiabilidad

Escenario de calidad 2. El sistema debe permitir cargar archivos de gran tamaño sin fallar durante el proceso.

- Fuente: Usuario.
- Estímulo: Cargar archivo de entrenamiento.
- Artefacto: Aplicación desarrollada.
- Ambiente: En tiempo de ejecución.
- Respuesta: El sistema debe poder responder a la carga de archivos de gran tamaño.
- Medida Rta: Tiempo de carga del archivo.

Al momento de desarrollar la interfaz de entrenamiento, se contaba con las opciones de realizarlo mediante una aplicación web, o una interfaz más sencilla en Java. Esta última fue la implementada, debido a que la aplicación web debe enviar al servidor correspondiente el contenido del archivo completo, ya que por cuestiones de seguridad no se puede obtener la ruta del mismo. Entonces, como el entrenamiento trata de archivos de gran tamaño, lo más probable es que la carga falle, no permitiendo cumplir con la fiabilidad.

Desarrollando la interfaz en Java, se logró evitar estas fallas, ya que se puede obtener el path del archivo directamente, y ser cargado al momento que se requiera.

4.4.3 Precisión

Escenario de calidad 3. La herramienta debe lograr un cierto grado de precisión aceptable en el modelo a construir.

- Fuente: Externa o Interna.
- Estímulo: Entrenar.
- Artefacto: Aplicación desarrollada.
- Ambiente: En tiempo de ejecución.

- Respuesta: El sistema debe generar un modelo con buena precisión sobre los datos.
- Medida Rta: Porcentaje correctos de clasificación y Recall.

Para lograr el nivel de precisión pretendido en el trabajo, se propuso el enfoque presentado en el capítulo anterior, el cual mejoró hasta obtener el enfoque de clasificación final, que con la aplicación de filtros logró porcentajes de aciertos elevados con respecto al resto.

En la Figura 4.4 se presenta el flujo relacionado a esta funcionalidad en un diagrama de secuencia:

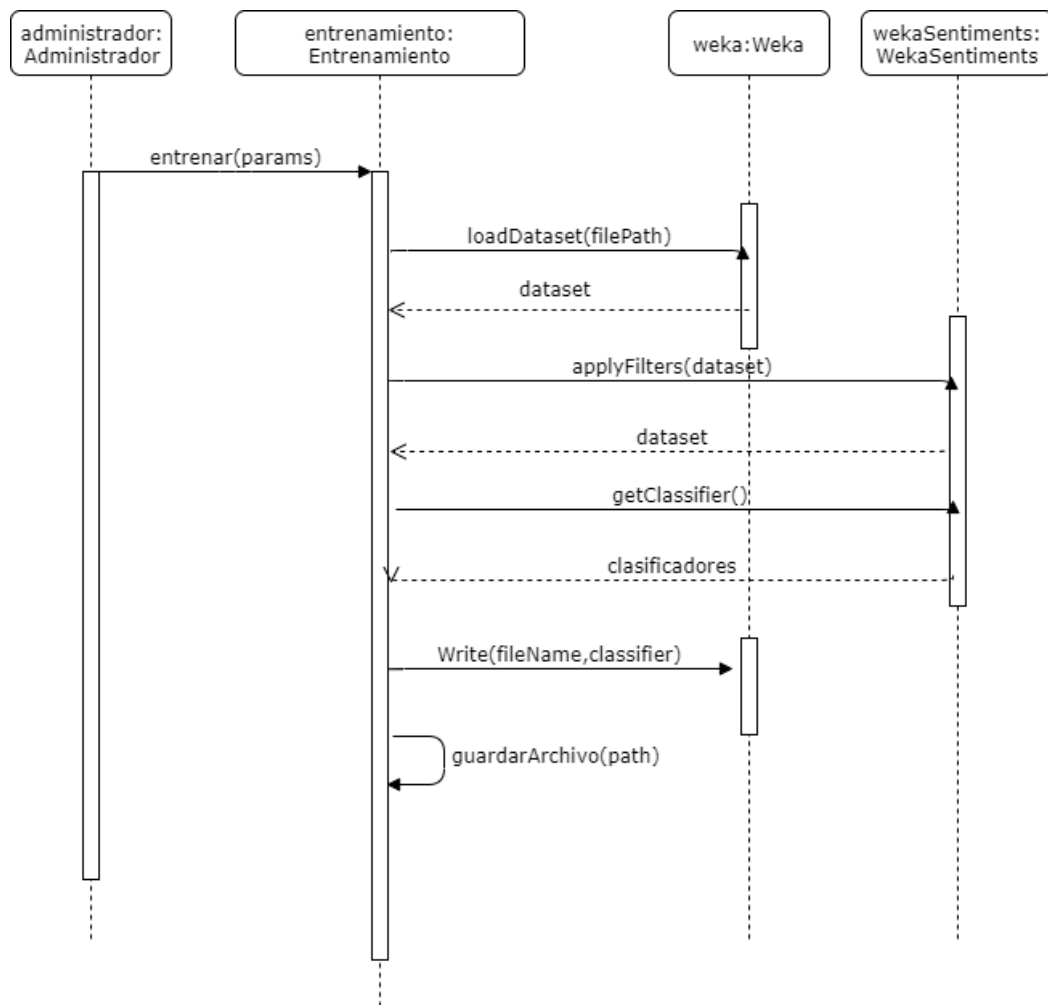


Figura 4.4. Diagrama de secuencia de entrenamiento

En este caso, una vez obtenido el dataset parseado se procede a aplicar los filtros `stringToWordVector`, `stemming` y `discretize`, se envía como parámetro el dataset.

Una vez obtenido el dataset filtrado, se realiza el entrenamiento normal, almacenando los resultados (`guardarArchivo`) y los modelos (llamando a `write` de Weka) correspondientes para futuros usos.

4.4.4 Performance

Escenario de calidad 4. La herramienta debe performar en cierto grado de aceptación, dado que la creación de modelos no deba durar un largo plazo.

- Fuente: Externa o Interna.
- Estímulo: Entrenar.
- Artefacto: Aplicación desarrollada.
- Ambiente: En tiempo de ejecución.
- Respuesta: El sistema debe crear modelos con una buena performance.
- Medida Rta: Tiempo de entrenamiento.

Al trabajar con la creación de modelos, un aspecto no menor es la performance, ya que a medida que se va incrementando su complejidad, está muy posiblemente afecte tanto a la calidad como al rendimiento, y es aquí donde hay un trade-off. Es por esto, que el modelo recibe algunas limitaciones, ya que se podría agregar variables específicas con las cuales se mejoraría la precisión pero aumentaría el grado de complejidad y en definitiva bajaría la performance, donde no se desea que el usuario demore días en crear el modelo y tenga que contar con un buen hardware para poder utilizar la herramienta.

4.4.5 Portabilidad

Escenario de calidad 5. El sistema debe funcionar correctamente en cualquier sistema operativo donde desee ejecutarlo.

- Fuente: Usuario.
- Estímulo: Correr la aplicación en otro sistema operativo diferente de Windows.
- Artefacto: Aplicación desarrollada.
- Ambiente: En tiempo de ejecución.
- Respuesta: El sistema debe funcionar con normalidad.
- Medida Rta: La portabilidad no debe afectar a la performance de la aplicación.

La aplicación de entrenamiento está desarrollada en Java, por lo que su ejecutable es un archivo JAR (Java ARchive), el cual se puede ejecutar en cualquier computadora de cualquier Sistema Operativo que contenga la máquina virtual de Java.

4.4.6 Modificabilidad

Escenario de calidad 6. El sistema debe proporcionar la posibilidad de modificarse fácilmente si se quieren agregar nuevos atributos.

- Fuente: Usuario.
- Estímulo: Cargar nueva información.
- Artefacto: Aplicación desarrollada.
- Ambiente: En tiempo de ejecución.
- Respuesta: El sistema debe mantener su performance a pesar de la nueva información.
- Medida Rta: Performance y posibilidad de escalar el conjunto de datos.

La aplicación se diseñó de tal forma que solo sea necesario modificar el pre procesamiento para agregar nueva información. En caso de querer adicionar atributos para inferir las tendencias, se debe contar con el dataset correspondiente del cual se puedan extraer los datos, modificar el header del archivo de entrenamiento para que lo reconozca. Luego el entrenamiento y la predicción van a tomar automáticamente esos nuevos atributos, incorporándolos al análisis.

Además, al utilizar weka, se posee la posibilidad de utilizar algoritmos que escalan fácilmente, por ejemplo SGD. Este permite agregar nuevos atributos y continuar funcionando normalmente ante la falta de estos, debido a que reemplaza globalmente todos los valores perdidos y transforma los atributos nominales en valores binarios. Además normaliza todos los atributos, por lo que los coeficientes en la salida se basan en los datos normalizados. Este algoritmo no fue usado en la solución, pero puede ser implementado en un escenario de modificabilidad.

4.5 Persistencia de los datos

A medida que se van generando las diferentes etapas del proceso, se van creando diferentes archivos ARFF que pueden ser consultados y analizados por el usuario.

4.6 Resumen

En esta sección, se presentó el diseño y la implementación de la herramienta. Se presentaron los distintos requisitos funcionales y no funcionales detectados, junto con los atributos de calidad.

Para un mejor entendimiento de lo que realiza la herramienta se agregaron distintos diagramas UML, tanto estáticos como diagramas de clase, de componentes, y de caso de uso, como dinámicos, como y diagramas de secuencia.

Se desarrollaron distintos escenarios de calidad, para visualizar cómo se cumplen los atributos de calidad establecidos. Estos atributos de calidad son: usabilidad, fiabilidad, precisión, escalabilidad, performance y portabilidad.

La herramienta está desarrollada bajo el lenguaje JAVA, y va a estar formada por un parser, que va a transformar el dataset para que quede usable de acuerdo a las necesidades, un organizador, que va a resumir los archivos pre procesados para obtener un único archivo de entrenamiento de cada tipo con los features a analizar, y distintas clases que van a entrenar a la herramienta.

5. Diseño e implementación de la aplicación de análisis de datos

En este capítulo se presenta la implementación de la aplicación mediante la cual el administrador va a poder realizar el análisis de datos e inferencia de las tendencias, junto con las decisiones de diseño. Va a estar dividido en diferentes secciones, de acuerdo a las funcionalidades descritas, como lo son los requisitos funcionales y atributos de calidad, la arquitectura, y los distintos escenarios de calidad que cumplen con los atributos.

5.1 Requisitos funcionales y atributos de calidad

Con toda la información recabada acerca de la herramienta a desarrollar es posible definir los requisitos funcionales del sistema de análisis de datos, los cuales van a permitir concretar todos los servicios que la misma debe cumplir, junto con las relaciones de su operación e implementación.

Entre los requisitos detectados se encuentran:

- La herramienta debe permitir que los usuarios puedan cargar tweets para analizar las tendencias.
- La herramienta debe permitir a los usuarios visualizar los resultados de forma clara, y a su vez tener la posibilidad de poder ver detalles de los mismos.
- La herramienta debe poder utilizarse sin la necesidad de instalación.

Por otra parte, se propone que la herramienta cumpla con los siguientes atributos de calidad, como características no funcionales deseables para el sistema:

- Performance: el tiempo que tarda el sistema en responder a un evento debe ser considerablemente corto, dentro de los parámetros de la herramienta, considerando que a veces se pueden procesar entradas de gran tamaño.
- Usabilidad: La herramienta debe ser simple para usar, permitiendo que gente ajena a la tecnología tenga la posibilidad de utilizarla.
- Portabilidad: Se requiere que la aplicación pueda ejecutarse en diferentes plataformas, es decir, que el código fuente sea capaz de reutilizarse sin necesidad de retocar el código.
- Disponibilidad: El sistema debe mantenerse operativo cuando se lo necesite y debe poder recuperarse fácilmente.
- Extensibilidad: Se deben poder agregar nuevos servicios a la aplicación si así se requiere.

Especificación de requerimientos

A continuación se describen de forma general los requerimientos de la aplicación de análisis de datos a través del diagrama de Casos de Uso de la Figura 5.1, los cuales van a ser detallados luego:

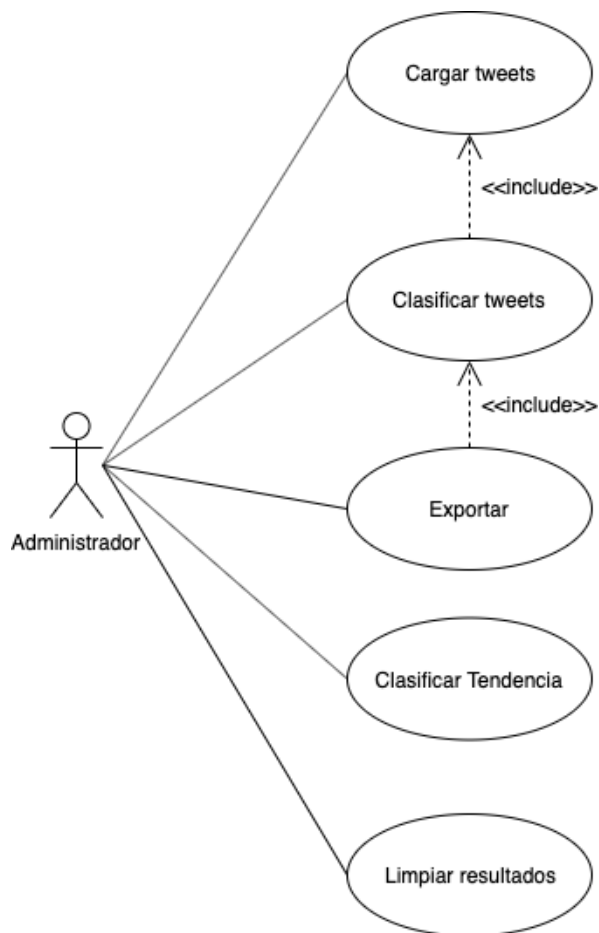


Figura 5.1. Diagrama de casos de uso aplicación de predicción.

1. **Cargar tweets:** El usuario debe poder cargar el archivo de tweets que desee para clasificar los sentimientos de los mismos.
2. **Clasificar tweet:** El usuario debe poder elegir los tweets que desea clasificar, y solicitarle al sistema que efectúe el análisis, obteniendo los resultados de forma clara y con la información necesaria.
3. **Exportar:** El usuario debe poder almacenar los resultados para que los mismos persistan.
4. **Clasificar tendencia:** El usuario debe poder ver reflejada la tendencia según la ventana de tiempo cargada.
5. **Limpiar resultados:** El usuario debe tener permitido reutilizar todas los tweets para analizarlas con otro clasificador. Para esto se debe contar con un botón que limpie los resultados del análisis previo.

5.2 Decisiones de diseño

En esta sección, se van a presentar las decisiones de diseño tomadas para que la aplicación pueda satisfacer los requisitos funcionales y los atributos de calidad presentados anteriormente.

De acuerdo a las características de la solución, se optó por diseñar una aplicación con una arquitectura Cliente-Servidor. La misma está formada por dos capas, la Capa de Presentación formada por la interfaz con la que el usuario va a interactuar y la Capa de Negocio, la cual se va a encargar del todo el procesamiento que abarca la predicción de tendencias, como se puede observar en la Figura 5.2.

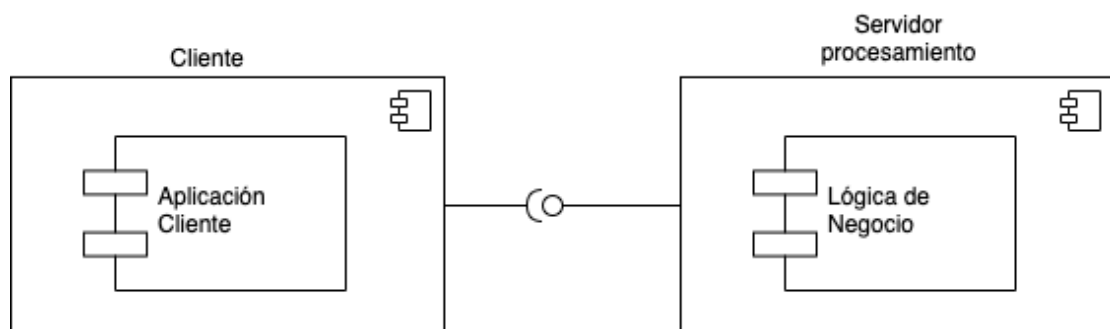


Figura 5.2. Arquitectura de la aplicación de análisis de datos

Aplicando la arquitectura Cliente - Servidor, se logra evitar que los usuarios tengan que llevar a cabo toda la instalación de las librerías, ya que el Servidor va a suplir esta necesidad, debido a que contiene todo el entorno preparado para ser utilizado.

Por otra parte, se optó por una aplicación debido a que hoy en día, con la presencia de numerosos frameworks, es mucho más sencillo desarrollar una aplicación de este tipo, antes que una interfaz de usuario con el idioma de programación JAVA, la cual era otra de las opciones.

5.2.1 Capa de presentación

Para realizar la interfaz de usuario se decidió emplear una tecnología Frontend que complemente y facilite el desarrollo.

Se optó por elegir JAVA por conocimientos y familiarización con el lenguaje utilizando Swing.

Swing es una librería gráfica para Java, la cual incluye widgets para interfaz gráfica de usuario tales como cajas de texto, botones, listas desplegables y tablas.

Además, esta librería sigue el modelo de arquitectura MVC para desarrollar interfaces gráficas para Java con independencia de la plataforma. Sigue un simple modelo de programación por hilos, y posee las siguientes características principales:

- Independencia de plataforma.
- Extensibilidad: es una arquitectura altamente particionada: los usuarios pueden proveer sus propias implementaciones modificadas para sobrescribir las implementaciones por defecto. Se puede extender clases existentes proveyendo alternativas de implementación para elementos esenciales.
- Personalizable: dado el modelo de representación programático del framework de Swing, el control permite representar diferentes estilos de apariencia "look and feel" (desde apariencia MacOS hasta apariencia Windows XP, pasando por apariencia GTK+, IBM UNIX o HP UX, entre otros). Además, los usuarios pueden proveer su propia implementación de apariencia, que permitirá cambios uniformes en la apariencia existente en las aplicaciones Swing sin efectuar ningún cambio al código de aplicación.

Entre sus principales ventajas se tienen:

- El diseño en Java puro posee menos limitaciones de plataforma.
- El desarrollo de componentes Swing es más activo.
- Los componentes de Swing soportan más características.

5.2.2 Capa de Negocio

Esta capa contiene toda la lógica utilizada para poder llevar a cabo la predicción de las tendencias. La misma está contenida en un Servicio Rest implementado en el lenguaje de programación Java. Esta arquitectura define una interface de comunicación entre cliente y servidor, separando completamente las responsabilidades entre ambas partes, y no mantienen estado asociado al cliente, permitiendo que cada petición que se realiza sea completamente independientes a la siguiente. Además, mantener un estado requiere memoria donde guardar todos los estados de todos los clientes, por lo que los servicios Rest son de gran ventaja para esto.

Al momento de elegir el servidor sobre el cual se iba a desplegar la aplicación se optó por Apache Tomcat ya que viene embebido en el Framework Spring Boot.

La funcionalidad del servicio se orientó a la predicción de las tendencias, separando la lógica del entrenamiento. De esta forma quedaron delimitadas dos partes de la solución al problema planteado: el entrenamiento de los clasificadores para la obtención de los modelos, y la predicción a partir de esos modelos.

5.3 Atributos de calidad

5.3.1 Usabilidad

Escenario de calidad 1. El usuario debe poder obtener fácilmente el archivo que contiene los tweets que desea procesar y poder realizar la carga del mismo.

- Fuente: Usuario.
- Estímulo: Cargar archivo de tweets.
- Artefacto: Aplicación desarrollada.
- Ambiente: En tiempo de ejecución.
- Respuesta: El sistema debe generar el archivo en formato arff.
- Medida de respuesta: Tiempo de carga del archivo.

La interacción del usuario con la aplicación comienza con una pantalla donde se puede cargar el primer archivo para comenzar a trabajar. Debido a que la herramienta de interés para proveer los tweets es Tweeter, como entrada se aceptan solo archivos .json, ya que es la extensión con la que se obtienen los tweets de la misma.

En la Figura 5.3 muestra esta pantalla inicial en la cual el usuario deberá hacer click en el boton “JsonToArff (Ranker)”.

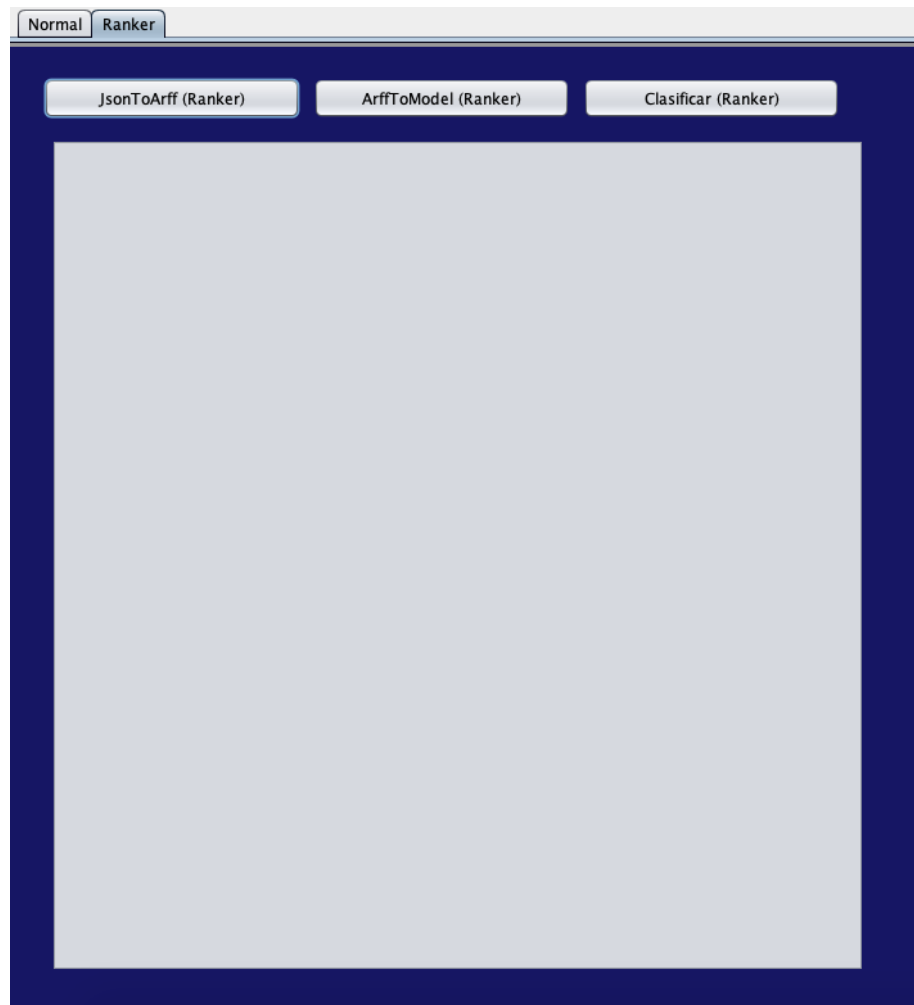


Figura 5.3. Pantalla inicial

Luego de clicar en ese botón, se abrirá una ventana de carga como se observa en la figura 5.4. Permitiendo al usuario seleccionar el archivo con los tweets en formato JSON.

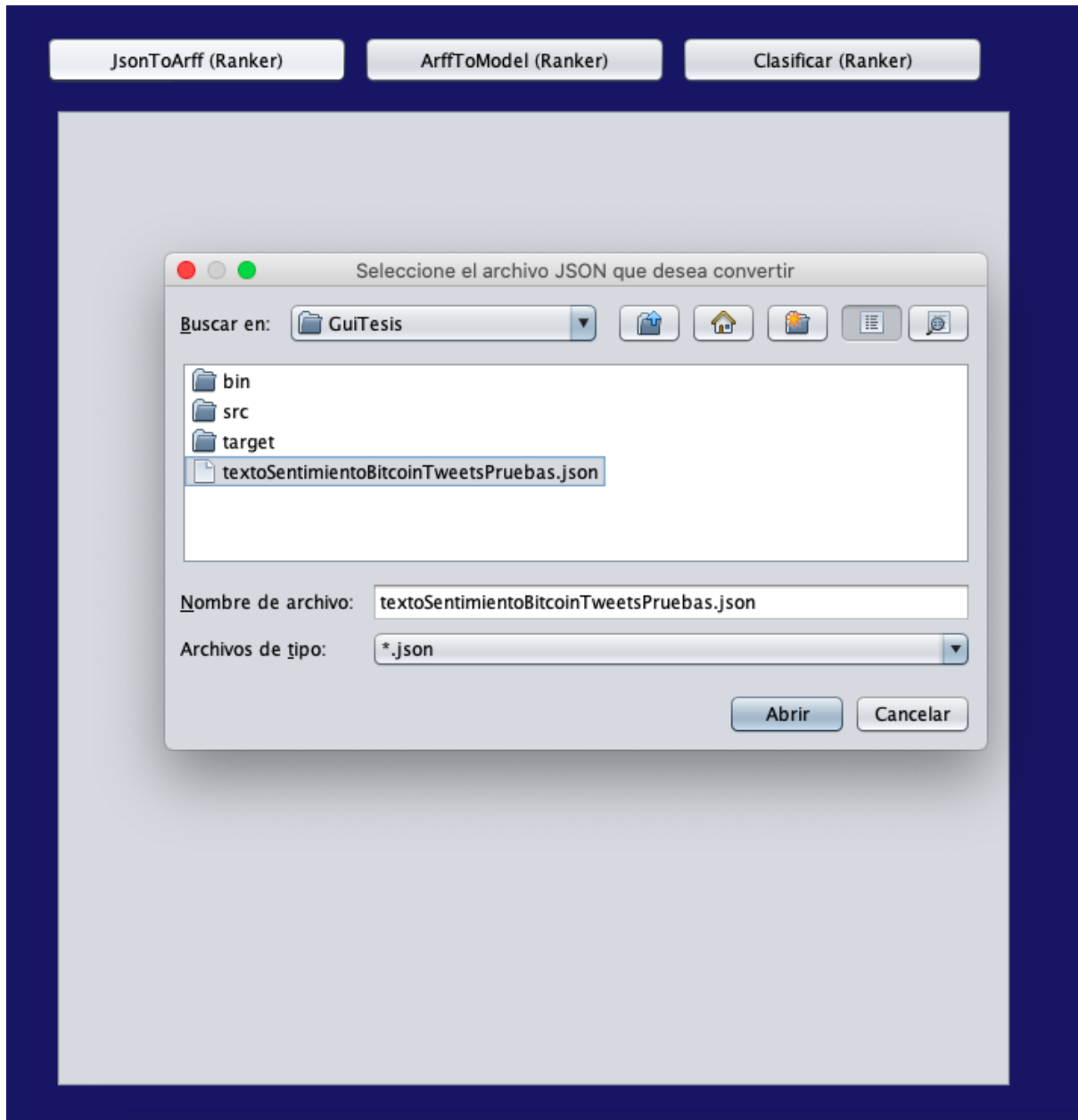


Figura 5.4. Ventana de carga para archivos en formato “json”.

Una vez seleccionado el archivo y dando click en el botón “Abrir”, se generará el archivo en formato arff.

Escenario de calidad 2. El usuario debe poder generar fácilmente modelo de clasificación a partir de un archivo en formato arff.

- Fuente: Usuario.
- Estímulo: Cargar archivo en formato arff.
- Artefacto: Aplicación desarrollada.
- Ambiente: En tiempo de ejecución.
- Respuesta: El sistema debe generar los modelos de clasificación.
- Medida de respuesta: Tiempo de carga del archivo.

La funcionalidad para generar modelos de clasificación está disponible para generar archivos en formato .model al momento de que el usuario hace click en el botón “ArffToModel (Ranker)” como se puede observar en la Figura 5.5.

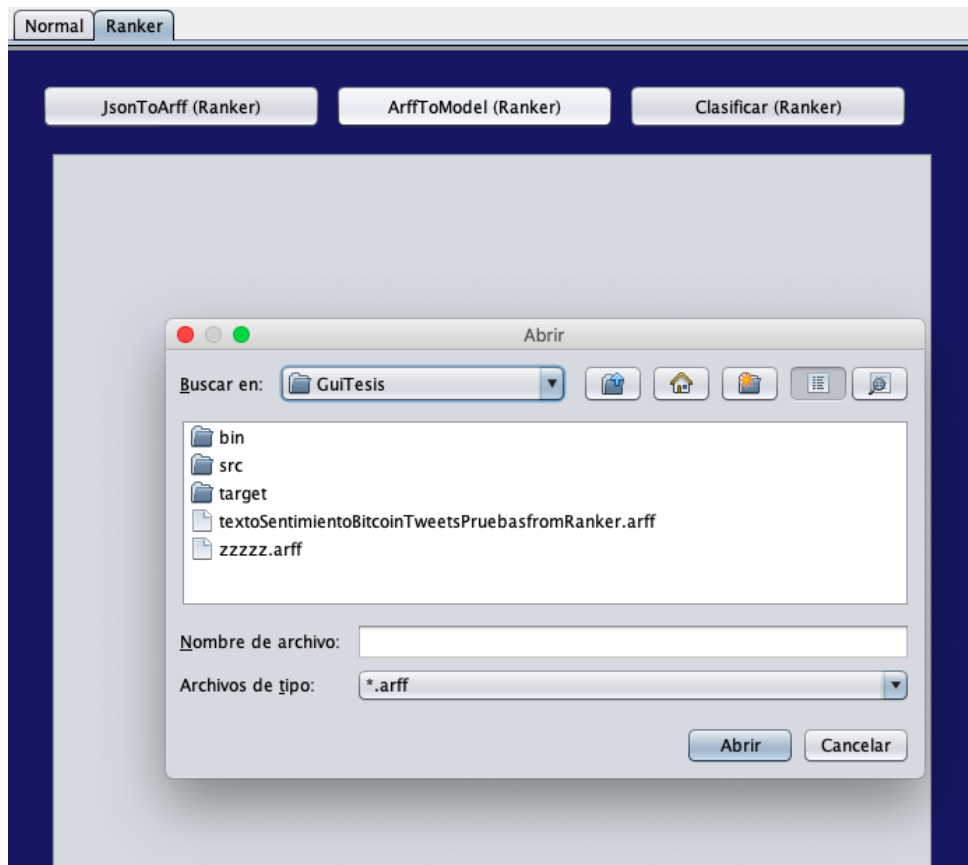


Figura 5.5 .Ventana de carga para archivos en formato “arff”.

Una vez cargado el archivo, el sistema generará los archivos .model correspondientes.

Escenario de calidad 3. El usuario debe poder clasificar fácilmente los tweets en formato JSON.

- Fuente: Usuario.
- Estímulo: Cargar archivo en formato json.
- Artefacto: Aplicación desarrollada.

- Ambiente: En tiempo de ejecución.
- Respuesta: El sistema debe clasificar las instancias.
- Medida de respuesta: Tiempo de carga del archivo.

La funcionalidad para generar clasificar nuevas instancias está disponible al momento de que el usuario hace click en el botón “Clasificar (Ranker)”. Luego se abrirá una ventana de carga de archivo en formato “arff”, el cual debe ser el mismo que se utilizó para generar el modelo previamente. En la Figura 5.6 se observa la ventana de carga.

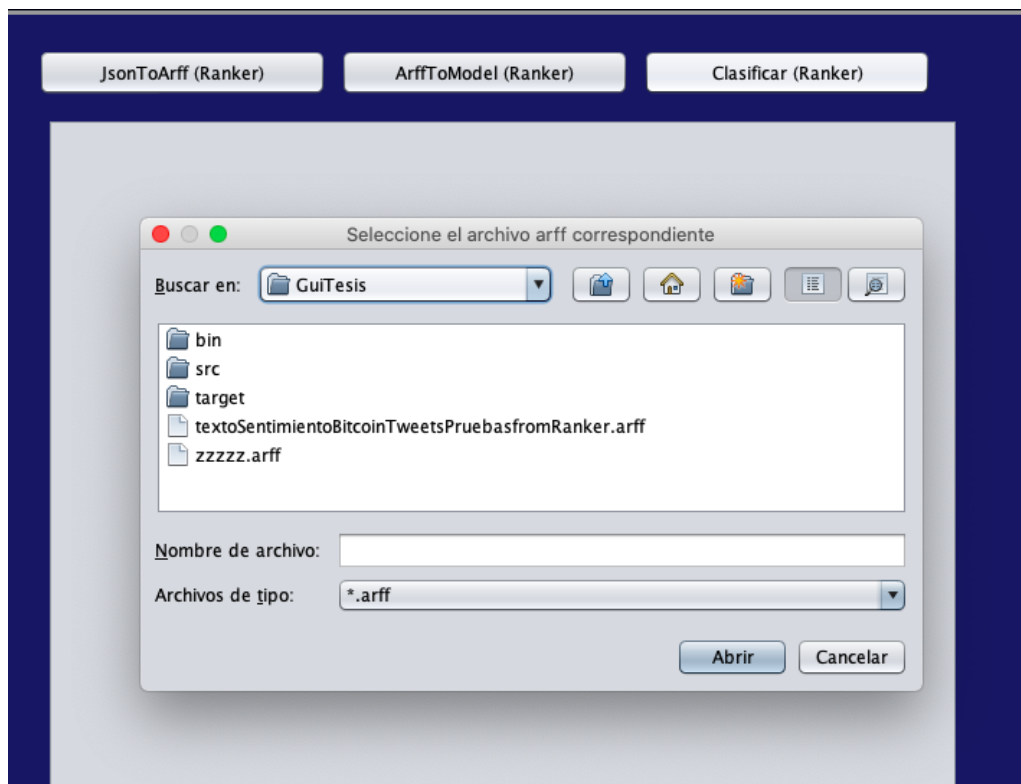


Figura 5.6 .Ventana de carga para archivos en formato “arff”.

Una vez cargado el archivo en formato “arff”, se procederá a la carga del archivo en formato “json” contenedor de las nuevas instancias que se desean clasificar. En la Figura 5.7 se observa la ventana de carga.

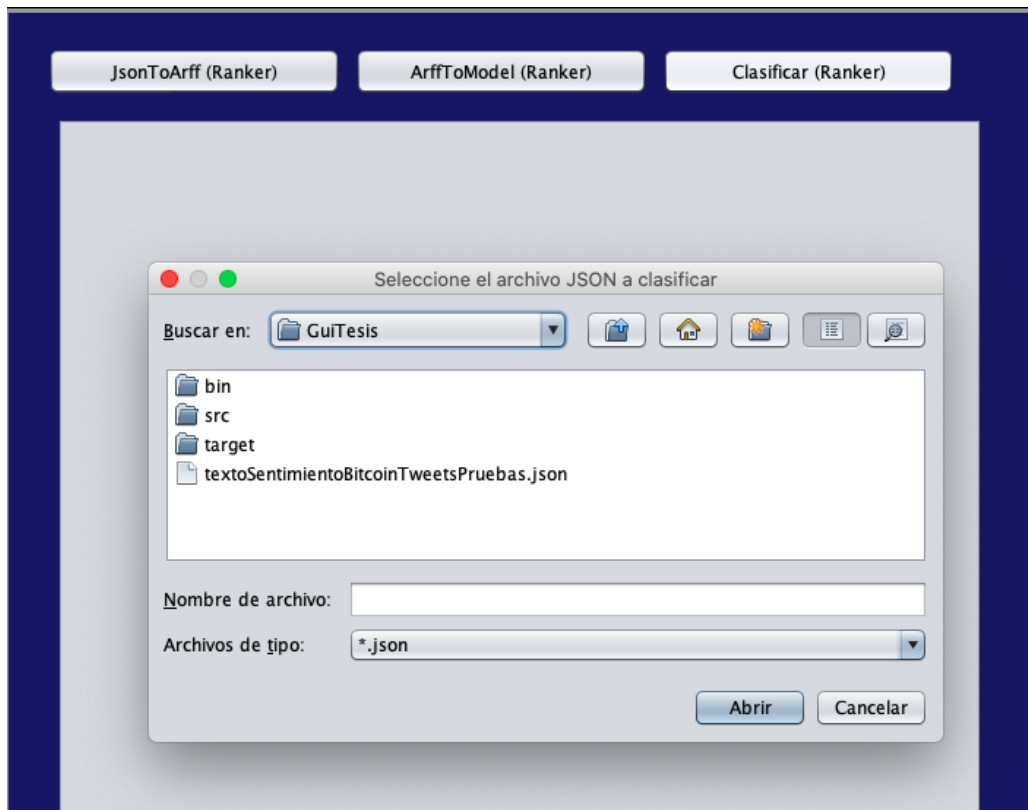


Figura 5.7 .Ventana de carga para archivos en formato “json”.

A continuación, se deberá cargar el clasificador deseado en formato “.model” como puede verse en la Figura 5.8.

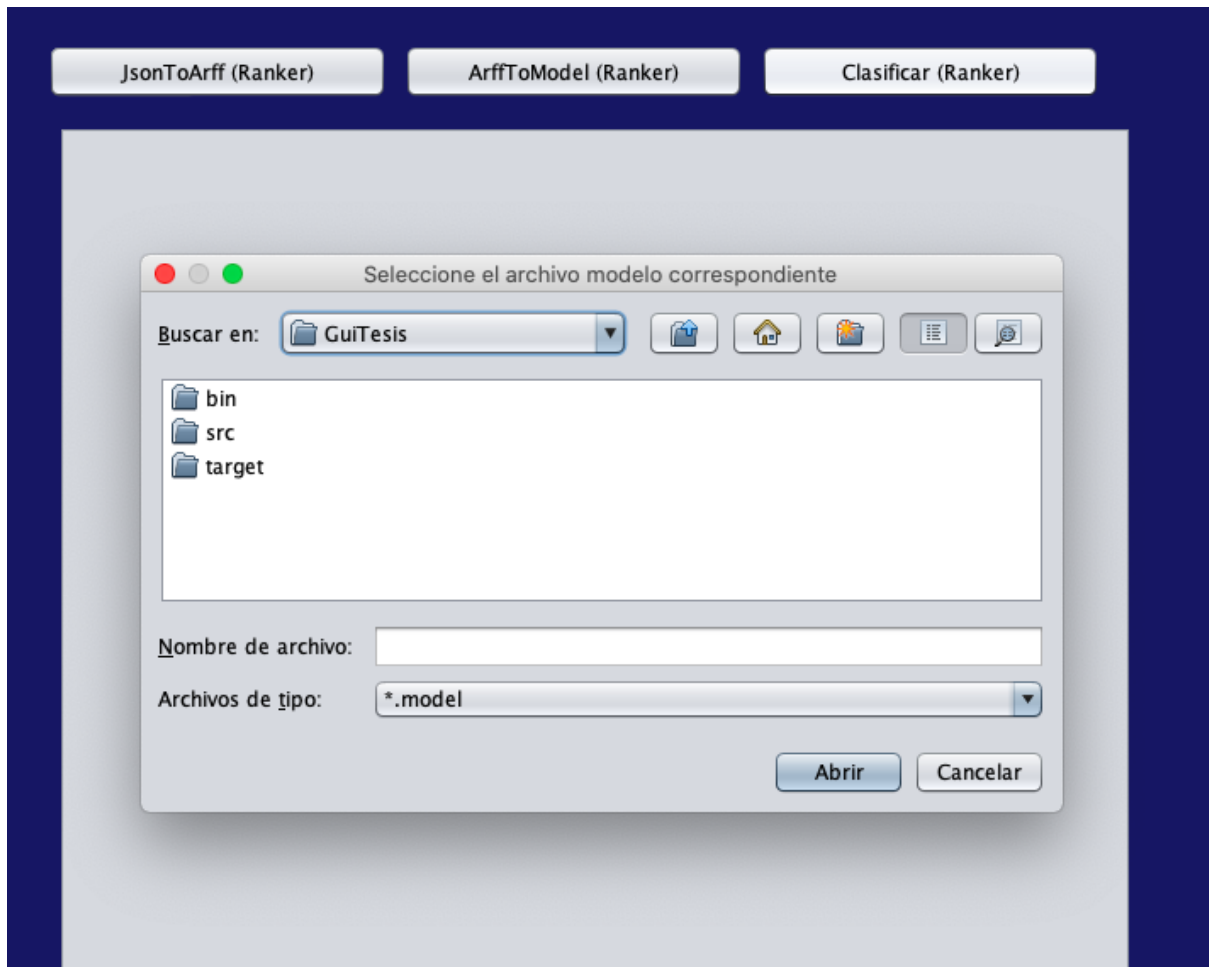


Figura 5.8 .Ventana de carga para archivos en formato “model”.

Una vez finalizado, el sistema generará como salida la cantidad de instancias negativas, positivas y neutrales.

5.3.2 Portabilidad

Escenario de calidad 4. Se desea que el sistema pueda ser utilizado en cualquier Sistema Operativo sin la necesidad de instalaciones previas.

- Fuente: Administrador.
- Estímulo: Utilizar la aplicación en varios SO.
- Artefacto: Aplicación desarrollada.
- Ambiente: En tiempo de ejecución.
- Respuesta: El sistema debe funcionar en cualquier SO donde desee ser utilizado.
- Medida de respuesta: El agregado de portabilidad no debe intervenir con el normal funcionamiento de la plataforma.

Para dar respuesta a este escenario, se diseñó a la Aplicación con una arquitectura Cliente Servidor, como se dijo anteriormente.

Swing, va a funcionar de igual manera cualquier Sistema Operativo, incluyendo SO móviles. Por su parte, la presencia del Servidor va a permitir tener configuradas todas las librerías sin necesidad de tener que cargar sus dependencias de acuerdo a cada sistema operativo, va a estar todo centralizado en el mismo.

5.3.3 Disponibilidad

Escenario de calidad 5. Se desea que el sistema esté operativo cuando se lo necesite y que en caso de falla se pueda restaurar fácilmente.

- Fuente: Interna o externa.
- Estímulo: Defecto, respuesta incorrecta.
- Artefacto: Aplicación desarrollada.
- Ambiente: En tiempo de ejecución.
- Respuesta: El sistema debe detectar la falla, registrarla y volver a estar operativo.
- Medida de respuesta: Tiempo para reparar la falla.

La decisión de separar a los tweets para ser procesados uno a uno también se tomó para evitar fallas que puedan alterar el correcto funcionamiento de la aplicación. Si se envía un archivo de gran tamaño (ya que no se puede enviar el path del mismo por cuestiones de seguridad), crece la probabilidad de que falle el sistema debido que se debe realizar la carga en el servidor y esto es muy costoso.

Por otra parte, la aplicación va a desplegarse en un contenedor Docker, el cual permite acelerar el tiempo promedio para la resolución de problemas. Esto se debe a que permite volver a desplegar fácilmente e implementar correcciones en la aplicación.

5.3.4 Escalabilidad

Escenario de calidad 5. Se desea agregar nuevos servicios a la aplicación, junto con una conexión a una base de datos.

- Fuente: Administrador.
- Estímulo: Agregar servicios.
- Artefacto: Aplicación desarrollada.
- Ambiente: En tiempo de compilación.
- Respuesta: El sistema debe continuar funcionando de forma similar a pesar del agregado de carga.
- Medida de respuesta: Tiempo de carga y respuesta.

Para dar respuesta a este escenario se cuenta con la arquitectura Cliente - Servidor. La estructura inherentemente modular facilita la integración de nuevas tecnologías y el crecimiento de la infraestructura computacional, favoreciendo así la escalabilidad de las soluciones. Se pueden agregar las capas que se deseen, como una capa de datos mediante una conexión a una base de datos, sin afectar el funcionamiento de la aplicación.

Los sistemas Cliente - Servidor pueden ser escalados horizontal o verticalmente. El escalado horizontal significa añadir o eliminar estaciones clientes con un ligero impacto en el rendimiento. A su vez, el escalado vertical significa la migración a un servidor más grande y rápida o la incorporación de nuevos servidores.

5.4 Persistencia de datos

Durante el desarrollo de la aplicación se analizó la forma en la que los datos iban a persistir. Se contaba con la opción de una Base de Datos, o de la descarga directa de los archivos con los resultados.

Se terminó optando por no levantar una Base de Datos, debido a que sólo se iba a utilizar para almacenar los resultados. Se consideró mejor la opción de que el usuario elija qué archivos quiere almacenar y en qué formato, de acuerdo al uso que le va a aplicar a los datos.

Para satisfacer esto se diseñó la funcionalidad de exportar. Con esto se busca que el usuario pueda almacenar sus resultados para futuros análisis. Se cuenta con las opciones de exportar a un archivo con el formato arff para poder ser reutilizado en la herramienta Weka.

6. Descripción del experimento

En esta sección se presenta una serie de análisis realizada sobre los atributos previo a presentar los resultados de las experimentaciones. Se realizó un experimento con técnicas de aprendizaje de máquina obteniendo las precisiones de los clasificadores. Por último, se utilizarán un conjunto de métricas para analizar el impacto de la aplicación.

6.1. Configuración

En esta etapa se procede a realizar una serie de análisis sobre atributos específicos y diferentes patrones que se visualizan en las tendencias. Este análisis de los datos permite identificar su validez e identificar cómo los algoritmos juzgan y obtienen su decisión. Además, permite mejorar la precisión no a partir de decisiones no intuitivas, sino a partir de los datos, de la información que nos otorga, tomando en fin mejores decisiones.

A su vez, se muestran diferentes tipos de gráficos para que el lector obtenga una representación más sencilla de lo que la información dispone y se puedan analizar aspectos interesantes de los datos.

Para introducir al lector en el experimento, es necesario hablar sobre el dataset de entrenamiento que se obtuvo a partir de la clasificación con GNL tras clasificar 18.181 instancias de las cuales resultaron 1.203 negativas, 11.379 neutrales y 5.599 positivas. En la Figura 6.1 se muestra como gráfico circular reflejando estos números en su proporción sobre el total.

Teniendo en cuenta esto, se busca tener un clasificador capaz de detectar el sentimiento de cada tweet para luego hacer un análisis cuantitativo de los mismo separándolos en ventanas de tiempo, en este caso de un día.

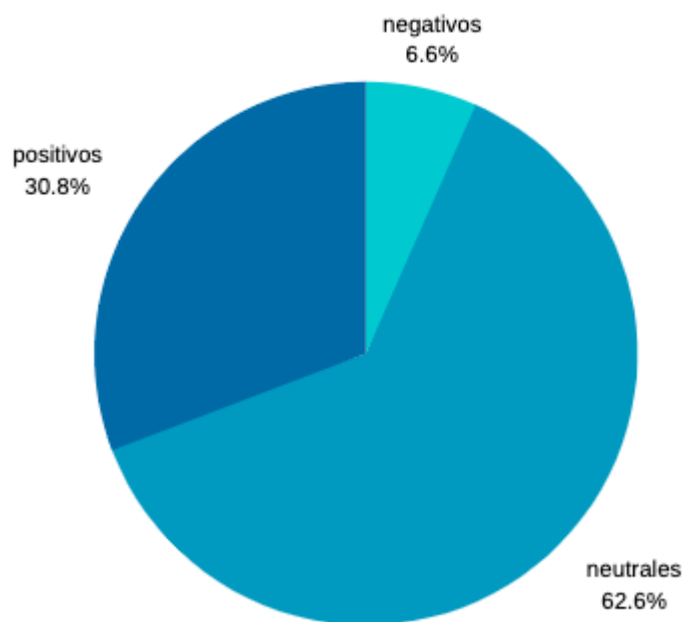


Figura 6.1

A simple vista puede notarse que el dataset está desbalanceado, por lo que se procedió a realizar un balanceo de los datos con la herramienta Weka utilizando el filtro supervisado por instancias “Class Balancer” para equilibrar el peso de los valores, obteniendo los resultados de la Figura 6.2.



Figura 6.2

Con esta nueva configuración, se procede a realizar el experimento como se detalla en las próximas secciones.

6.2. Resultados de experimentación con técnicas de aprendizaje de máquina

Debido a que el tipo de aprendizaje de máquina utilizado en este trabajo es de tipo supervisado, se contaba con un conjunto de datos previamente etiquetado, es decir, se tenía un conjunto de muestra, el cual ya se sabía a qué valor de clase pertenece cada registro. Cabe destacar que las etiquetas de estos datos, fueron provistos por GNL y pueden tener un error asociado, pero se tomaron estas etiquetas como válidas. Luego, comparando el resultado de las clasificaciones hechas sobre este dataset con los valores originales que contenía el mismo, se realizó la primer experimentación, obteniendo un conjunto de métricas.

En esta sección se van a presentar los resultados obtenidos de los distintos clasificadores, poniendo foco en aquellos que mejor precisión lograron. Se va a realizar un análisis general, y luego se los va a analizar individualmente interpretando los resultados obtenidos.

Las métricas más tradicionales para medir la eficacia de un modelo de aprendizaje son precisión y recall. Antes de definirlos, se deben dar a conocer algunos conceptos, que forman parte de su definición:

- **True Positives (TP) para la clase C:** son instancias pertenecientes a la clase C que se clasifican correctamente en la clase C.
- **True Negatives (TN) para la clase C:** son instancias no pertenecientes a la clase C y que no se clasifican como clase C.
- **False Positives (FP) para la clase C:** son instancias no pertenecientes a la clase C pero que se clasifican como clase C.
- **False Negatives (FN) para la clase C:** son instancias pertenecientes a la clase C pero que no se clasifican como clase C.

Una vez definidos estos conceptos, la precision y recall se calculan de la siguiente manera:

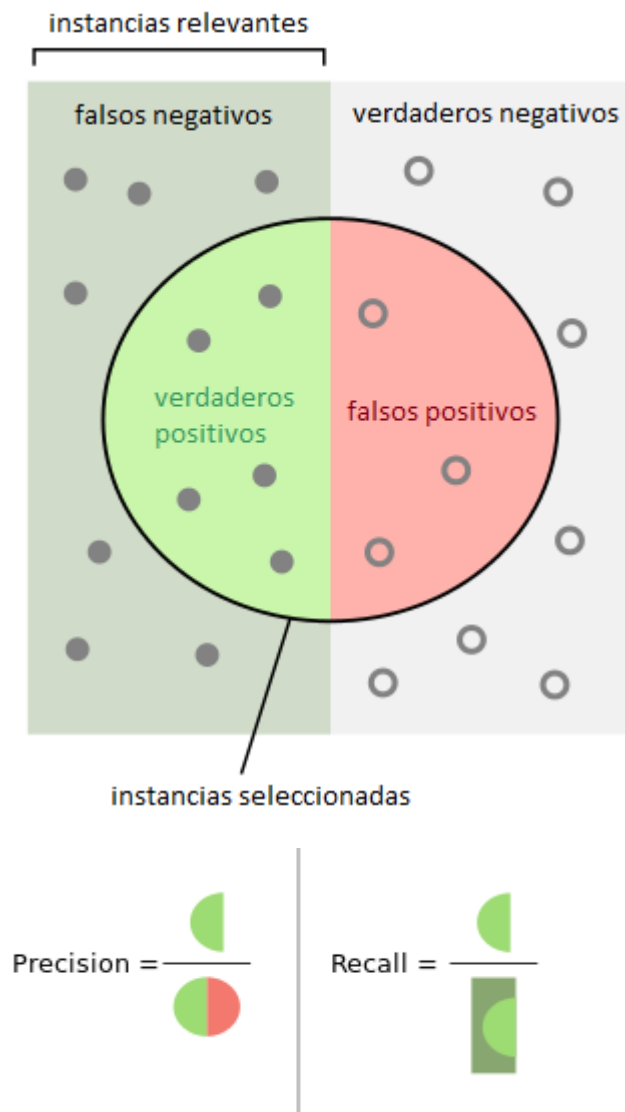
- **Precisión para la clase C:** es un valor entre 0 y 1. Su valor aumenta cuando hay pocos falsos positivos. Mide que las instancias clasificadas como clase C sean realmente de la clase C, aunque haya instancias de la clase C que se clasifiquen como otra clase.

$$P(C) = TP / (TP + FP)$$

- **Recall para la clase C:** es un valor entre 0 y 1. Su valor aumenta cuando hay pocos falsos negativos. Mide que las instancias de la clase C se clasifiquen como clase C, aunque otras instancias también se clasifiquen como clase C sin serlo.

$$R(C) = TP / (TP + FN)$$

Para graficar estos conceptos, se presentan las figuras



6.2.1 Análisis general clasificadores

En esta sección se describen los resultados de las pruebas realizadas sobre el enfoque propuesto en la sección 3 (Enfoque Propuesto) dentro del capítulo 3, “Solución Propuesta”, y se analiza el desempeño general.

Se probaron diferentes algoritmos de clasificación, utilizando 10-fold cross validation sobre el conjunto de datos de entrenamiento.

Clasificador	DecisionTable	NaiveBayes	SMO	J48
Instancias correctamente clasificadas	70.32 %	68.07 %	80.98 %	65.58 %

Como se observa, con estos datasets y con preprocesamiento, las precisiones obtenidas son aceptables.

A continuación se presenta la actuación individual de cada uno de los clasificadores que lograron la mayor precisión a la hora de clasificar a qué clase pertenece un determinado conjunto de datos. Para ello se propuso indicar las diferentes columnas TP Rate, FP Rate, Precision, Recall, F-Measure, MCC, ROC Area y RPC Area. Si bien cada una de estas columnas representa valiosa información acerca de las decisiones tomadas, el análisis se centrará principalmente en la Precisión, el Recall y F-Measure, ya que se considera, en conjunto a la matriz de confusión, que se da de forma sencilla y rápida un paneo general acerca de los resultados.

Este análisis en profundidad se realizará únicamente en aquellos clasificadores cuyo modelo fue el más exitoso, mientras que con los modelos clasificadores restantes solo se ilustrara los resultados obtenidos

6.2.2 Análisis individual clasificadores entrenamiento

Como primer algoritmo a presentar se encuentra el SMO, el cual obtuvo alrededor de un 5% de asertividad mayor que el promedio de los clasificadores y un 10,66% del siguiente mejor clasificador, Decision Table. Este algoritmo dentro de la librería se encuentra implementado para utilizar con conjuntos de datos multiclase, resolviendo cada clase como 1-vs-1, más conocido como “clasificación pairwise”, como puede analizarse en la Figura

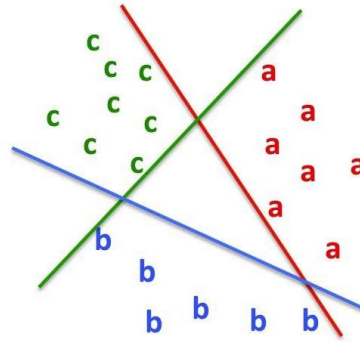


Figura Clasificación pairwise

De esta forma el algoritmo separa las diferentes clases y decide a qué clase pertenece cada caso. Donde para calcular esta clasificación utiliza el método estadístico de acoplamiento de Hastie y Tibshirani (Hastie, T., Tibshirani, R., & Friedman, J. , 2009), obteniendo los resultados presentados en la Tabla 6.1:

TP Rate	FP Rate	Precisión	Recall	F-Measure	MCC	ROC Area	PRC Area	Clase
0,796	0,029	0,932	0,796	0,858	0,801	0,911	0,850	Negativo
0,824	0,166	0,712	0,824	0,764	0,637	0,830	0,648	Neutral
0,810	0,090	0,818	0,810	0,814	0,722	0,897	0,751	Positivo
0,810	0,095	0,821	0,810	0,812	0,720	0,879	0,750	Avg.

Tabla 6.1. Clasificador SMO

Obteniendo la matriz de confusión que se observa en la Tabla 6.2.

Negativos	Neutrales	Positivos	Clase
9642.13	1894.17	584.37	Negativos
538.98	9987.11	1594.57	Neutrales
164.52	2136.65	9819.49	Positivos

Tabla 6.2. Matriz de confusión.

Como se puede ver al igual que el porcentaje de clasificación los resultados obtenidos a partir de las mediciones, son aceptables ya que la clasificación consta de tres clases.

Puede observarse que el valor de ROC Area es el que resalta respecto al resto, en los casos como por ejemplo el negativo donde fue una clase con mayor acierto que el resto.

6.3 Aplicación y evaluación de la herramienta resultante

En esta sección se evalúa la herramienta desarrollada en un caso de experimentación real, en el cual se toman como muestra el dataset antes mencionado con ventanas de clasificación por el intervalo de un día y sabiendo el resultado objetivo según los valores de mercado correspondiente al Bitcoin durante el periodo de tiempo comprendido entre [30/11/2017 - 14/02/2018].

Para evaluar la herramienta, se decidió un análisis cuantitativo sobre la cantidad de tweets positivos y negativos arrojados por el clasificador de este experimento, obviando los neutrales.

Utilizando como fuente de entrada el arff conteniendo los tweets ya clasificados por GNL, como primera instancia se seleccionó un conjunto de atributos para trabajar con el enfoque propuesto (Ranker).

Como primer paso para esta selección, fue necesario aplicar el filtro de atributos “StringToWordVector” para que luego las palabras puedan ser identificadas como atributos. Este filtro se configuró como se muestra en la Figura 6.3.

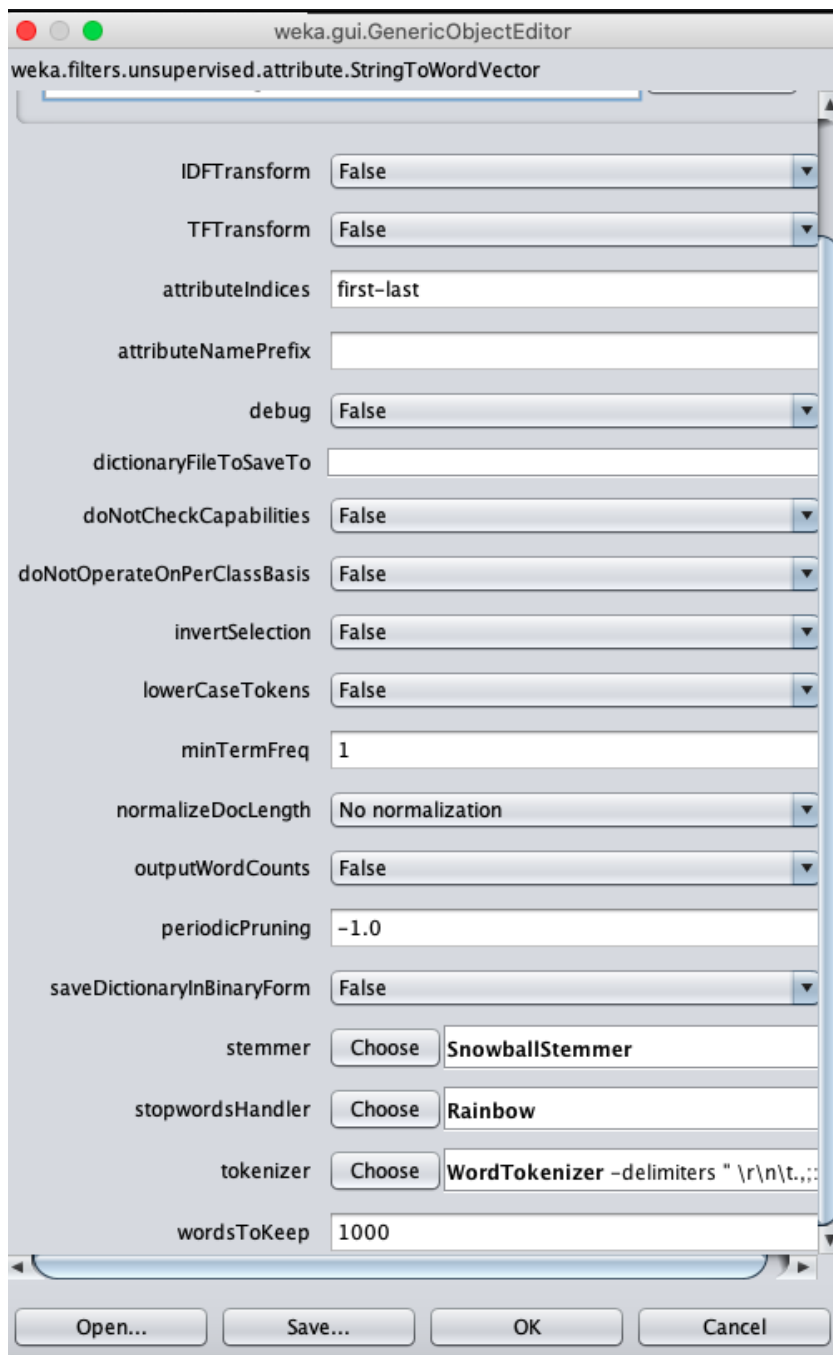


Figura 6.3.

Luego, se utilizó la herramienta Weka Greedy Stepwise como se muestra en la Figura 6.4 para generar la selección de atributos óptima.

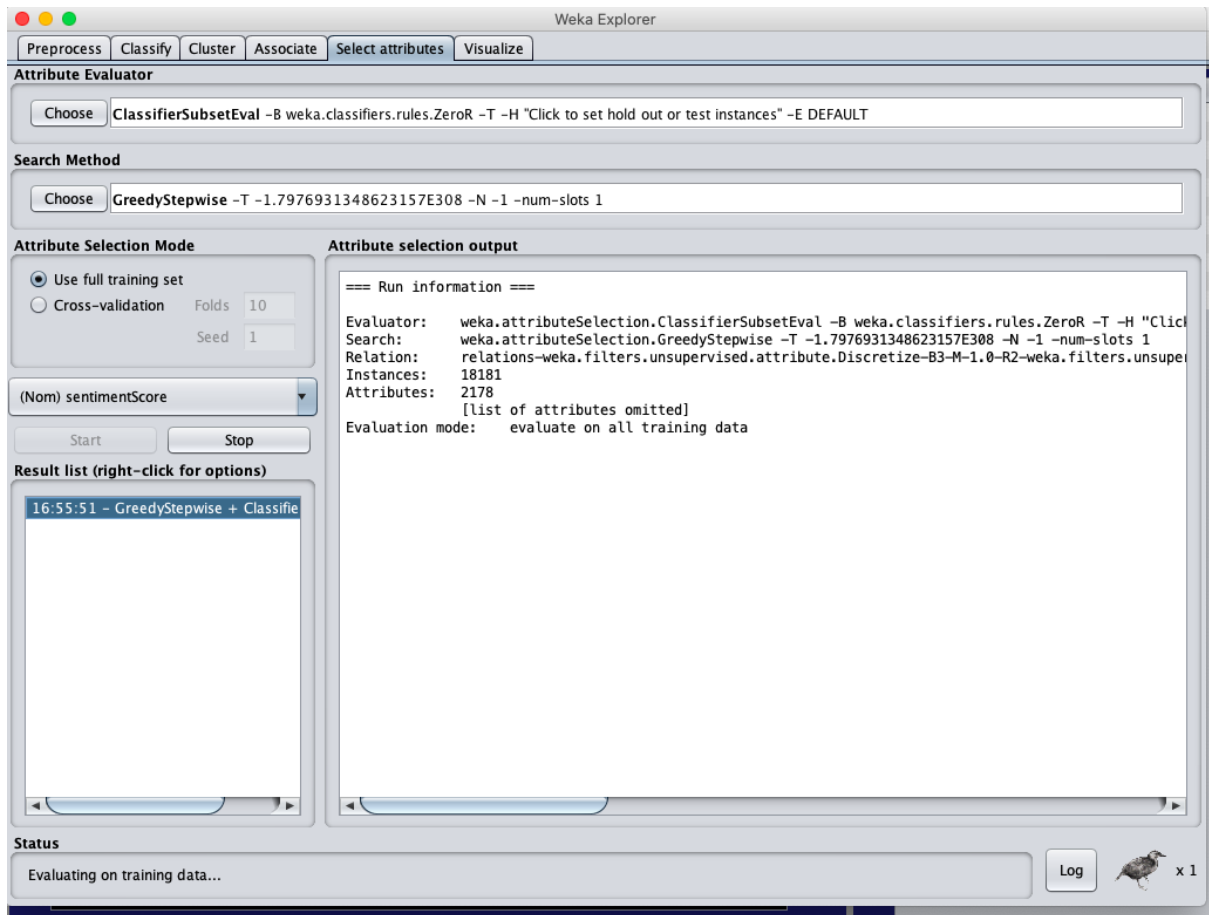


Figura 6.4.

Generando como resultado las features que se muestran en la tabla 6.3:

#bitcoin
#blockchain
#btc
#crypto
#cryptocurrency
\$btc
avoid
bank
bitcoin
cards
crypto

cryptocurrency
exchanges
fades
miners
stolen
visa
btc
mining
trading

Tabla 6.3

Una vez obtenidos los resultados, se procedió a experimentar con los datasets almacenados por el stream de twitter previamente mencionado.

Para esta parte se decidió automatizar el proceso mediante Test de Java siguiendo una serie de pasos como se detalla a continuación. La decisión de automatizar de esta forma, es para poder arrojar en un análisis masivo de los datos y agilizar los procesos de prueba.

Paso 1: Generación del nuevo arff a partir del archivo con los resultados arrojados por GNL en formato JSON siguiendo el enfoque Ranker, para ello se deben seleccionar sólo los features de la tabla como se muestra en la figura 6.5.

```

@Test
public void testJsonToArffRanker () throws Exception {
    ConverterImpl converterFromRanker = new ConverterImpl();
    List<String> features = new ArrayList<>();
    features.add("#bitcoin");
    features.add("#blockchain");
    features.add("#btc");
    features.add("#crypto");
    features.add("#cryptocurrency");
    features.add("$btc");
    features.add("avoid");
    features.add("bank");
    features.add("bitcoin");
    features.add("cards");
    features.add("crypto");
    features.add("cryptocurrency");
    features.add("exchanges");
    features.add("fades");
    features.add("miners");
    features.add("stolen");
    features.add("visa");
    features.add("btc");
    features.add("mining");
    features.add("trading");
    String pathJsonDataset = "/Users/juanezequielojeda/Documents/Repot/repositoriotesis/Tesis_v1.0/textoSentim
    converterFromRanker.jsonToArffFromRanker(features,pathJsonDataset);
}

```

Figura 6.5.

Paso 2: Generación modelos de clasificación a partir del archivo arff generado en el paso anterior como se muestra en la figura 6.6.

```

@Test
public void testModelFromArffRanker () throws Exception {
    ClasificadorCreatorImpl clasificadorFromRanker = new ClasificadorCreatorImpl();
    String pathDataset = "/Users/juanezequielojeda/Documents/pruebasTesis/textoSentimientoBitcoinTweetsfromRanker.arff";
    clasificadorFromRanker.modelCreatorFromRanker(pathDataset);
}

```

Figura 6.6.

Paso 3: Clasificación utilizando el arff generado en el primer paso, el modelo SMO y los archivos en formato JSON contenedores de los tweets a clasificar como se muestra en la figura 6.7.

```

@Test
public void clasificarDia() throws Exception {
    String pathModel = "/Users/juanezequielojeda/Documents/pruebasTesis/models/modelFromRankerSMO.model";

    ClasificadorTextSentimentWeka clasificador = new ClasificadorTextSentimentWeka();
    List<String> dias = new ArrayList<>();
    dias = getDias();

    String pathDataset = "/Users/juanezequielojeda/Documents/pruebasTesis/textoSentimientoBitcoinTweetsfromRankerDiscretizadoFromRanker.arff";
    Archivo logsTodos = new Archivo( nombreBase: "logsTodosSMO");

    for (String dia:dias){
        Archivo logs = new Archivo( nombreBase: "logs"+dia);

        int negativos = 0;
        int positivos = 0;
        int neutrales = 0;

        List<String> pathDias = pathDias(dia);

        for (String pathDia:pathDias) {
            HashMap hashMap = clasificador.clasificarFromRanker(pathModel, pathDia, pathDataset);

            System.out.println("POSITIVOS HASH: " + hashMap.get("positivos"));
            System.out.println("NEUTRALES HASH: " + hashMap.get("neutrales"));
            System.out.println("NEGATIVOS HASH: " + hashMap.get("negativos"));
            negativos += ((Integer) hashMap.get("negativos")).intValue();
            positivos += ((Integer) hashMap.get("positivos")).intValue();
            neutrales += ((Integer) hashMap.get("neutrales")).intValue();
        }
        System.out.println("");

        logsTodos.escribir( texto: "\n"+dia+", "+negativos+", "+neutrales+", "+positivos);
        logs.escribir( texto: "\nResultados del dia: "+dia);

        logs.escribir( texto: "\nNEGATIVOS: "+negativos);
        logs.escribir( texto: "\nPOSITIVOS: "+positivos);
        logs.escribir( texto: "\nNEUTRALES: "+neutrales);
        logs.cerrar();
    }
}

```

Results clasificarDia()

Figura 6.7.

Arrojando los resultados de la tabla 6.4 para el clasificador SMO

Fecha	Negativos	Neutrales	Positivos	Resultado obtenido	Resultado Esperado
30/11/2017	110075	19727	20666	Alza	Alza
1/12/2017	152308	24007	29185	Alza	Alza
2/12/2017	81084	12242	23127	Alza	Alza
3/12/2017	106418	23578	24345	Alza	Alza
4/12/2017	137979	32253	31064	Baja	Alza
5/12/2017	146199	29999	30232	Alza	Alza
6/12/2017	178698	42440	35632	Baja	Alza
7/12/2017	277893	203482	40543	Baja	Alza
8/12/2017	243153	237656	37521	Baja	Baja
9/12/2017	187160	94999	30128	Baja	Baja
10/12/2017	185852	67861	32363	Baja	Alza
11/12/2017	213037	53506	33663	Baja	Alza

12/12/2017	163838	41443	29475	Baja	Alza
13/12/2017	148666	43383	31579	Baja	Baja
14/12/2017	156985	45443	36562	Baja	Alza
15/12/2017	162033	45682	39223	Baja	Alza
16/12/2017	141157	43811	34129	Baja	Alza
17/12/2017	125711	30519	36974	Alza	Baja
18/12/2017	161865	34891	39504	Alza	Baja
19/12/2017	191709	37049	33654	Baja	Baja
20/12/2017	192559	49897	35160	Baja	Baja
21/12/2017	162721	41005	31595	Baja	Baja
22/12/2017	239080	62125	39161	Baja	Baja
23/12/2017	174085	50476	30898	Baja	Alza
24/12/2017	66949	24521	18479	Baja	Baja
25/12/2017	85147	39241	24411	Baja	Alza
26/12/2017	124113	28814	33027	Alza	Alza
27/12/2017	115989	25396	31085	Alza	Baja
28/12/2017	113531	34053	32098	Baja	Baja
29/12/2017	100831	29650	32299	Alza	Baja
30/12/2017	88465	28939	29632	Alza	Baja
31/12/2017	90332	23656	30115	Alza	Alza
1/01/2018	93572	21164	29359	Alza	Baja
2/01/2018	112721	35163	34973	Baja	Alza
3/01/2018	103099	28950	32320	Alza	Alza
4/01/2018	102360	26822	30147	Alza	Alza
5/01/2018	99773	17639	34707	Alza	Alza
6/01/2018	76618	17328	28590	Alza	Alza
7/01/2018	75566	15002	26096	Alza	Baja
8/01/2018	98456	21241	30888	Alza	Baja
9/01/2018	106053	29370	31217	Alza	Baja
10/01/2018	44772	12058	14541	Alza	Alza
11/01/2018	125714	45520	33065	Baja	Baja
12/01/2018	101953	43917	31301	Baja	Alza
13/01/2018	78111	30033	26464	Baja	Alza

14/01/2018	42776	8314	14856	Alza	Baja
15/01/2018	87785	15523	26267	Alza	Baja
16/01/2018	121833	36602	34672	Baja	Baja
17/01/2018	170306	76486	39722	Baja	Baja
18/01/2018	139662	36757	32719	Baja	Alza
19/01/2018	117587	31645	36003	Alza	Alza
21/01/2018	56797	14549	14140	Baja	Baja
22/01/2018	13043	2313	2959	Alza	Baja
23/01/2018	84371	17733	21785	Alza	Alza
24/01/2018	11864	6680	3362	Baja	Alza
27/01/2018	45939	12487	12762	Alza	Alza
28/01/2018	10506	2596	2111	Baja	Alza
29/01/2018	97118	16639	21398	Alza	Baja
31/01/2018	98598	22510	25672	Alza	Baja
1/02/2018	40725	17444	13084	Baja	Baja
2/02/2018	26627	9469	7119	Baja	Baja
10/02/2018	58355	5100	5692	Alza	Baja
11/02/2018	204757	18257	23284	Alza	Baja
12/02/2018	147937	17276	25490	Alza	Alza
13/02/2018	74452	8598	15065	Alza	Baja
14/02/2018	97260	13162	24349	Alza	Alza

Tabla 6.4.

De aquí se obtuvo la matriz de confusión que se muestra en la tabla 6.5.

	Negativos	Positivos
Negativos	15	17
Positivos	17	17
Total por clase	32	34

Tabla 6.5. Matriz de confusión.

Obteniendo las métricas que se muestran en la tabla 6.6.

TP Rate	FP Rate	Precisión	Recall	F-Measure	MCC	Clase
---------	---------	-----------	--------	-----------	-----	-------

0,4688	0,50	0,4688	0,4688	0,4688	0,0313	Negativo
0,50	0,4688	0,5313	0,50	0,5152	0,0313	Positivo
0,4844	0,4844	0,50	0,4844	0,492	0,0313	Avg.

Tabla 6.6

Obteniendo un TPR de 48.48%, si bien el clasificador por sí mismo obtiene buenos resultados (como se muestra en la Tabla 6.1), no se pudo demostrar que se cumple la hipótesis planteada inicialmente, la de poder predecir la tendencia del Bitcoin.

Adicionalmente, a continuación se detalla otro método en donde se aplica el método RandomUndersampler para el balanceo del dataset y SGD como clasificador

RandomUndersampler es una forma rápida y fácil de balancear el dataset mediante la selección aleatoria de un subconjunto de datos para las clases objetivo. El remuestreo con múltiples clases se realiza considerando de forma independiente cada clase objetivo. Además, RandomUnderSampler permite muestrear datos heterogéneos (por ejemplo, que contienen algunas cadenas).

Utilizando el método RandomUndersamplear para la selección y SGD para la clasificación se obtuvieron los resultados que se muestran a continuación.

TP Rate	Precisión	Recall	F-Measure	Clase
0.91	0.38	0.81	0.52	Negativo
0.83	0.92	0.70	0.79	Neutral
0.83	0.52	0.77	0.62	Positivo
0.83	0.80	0.72	0.74	Avg.

Tabla 6.7. Clasificador SGD.

Como se observa en la Tabla 6.7, los resultados desmejoraron con respecto a lo obtenido con SMO pero mejoraron en la aplicación real como se observa en la Tabla 6.8, obteniendo un TPR del 56% (mejorando el TPR de 48.48% que se detalla en 6.3).

Fecha	Negativos	Neutrales	Positivos	Resultado Obtenido	Resultado Esperado
20171130	18799	106775	24894	Alza	Alza
20171201	21828	147541	36131	Alza	Alza
20171202	11919	82800	21734	Alza	Alza
20171203	20060	108583	25698	Alza	Alza
20171204	22061	142731	36504	Alza	Alza
20171205	19275	150461	36694	Alza	Alza
20171206	32956	180413	43401	Alza	Alza
20171207	59710	392681	69527	Alza	Alza
20171208	65287	384245	68798	Baja	Baja
20171209	42828	218168	51291	Baja	Baja
20171210	29943	209456	46677	Alza	Alza
20171211	33126	220973	46107	Alza	Alza
20171212	27531	172185	35040	Alza	Alza
20171213	26605	157513	39510	Alza	Baja
20171214	25552	171504	41934	Alza	Alza
20171215	41310	155609	50019	Alza	Alza
20171216	30881	138440	49776	Alza	Alza
20171217	20012	130453	42739	Alza	Baja
20171218	23788	166494	45978	Alza	Baja
20171219	28306	185974	48132	Alza	Baja
20171220	28861	190056	58699	Alza	Baja
20171221	24981	169854	40486	Alza	Baja
20171222	49325	235597	55444	Alza	Baja
20171223	35060	173259	47140	Alza	Alza
20171224	13988	77486	18475	Alza	Baja
20171225	13975	108997	25827	Alza	Alza
20171226	18827	130256	36871	Alza	Alza
20171227	18207	122710	31553	Alza	Baja
20171228	24188	122754	32740	Alza	Baja
20171229	17277	112582	32921	Alza	Baja
20171230	13190	102987	30859	Alza	Baja

20171231	13530	101834	28739	Alza	Alza
20180101	13736	98087	32272	Alza	Baja
20180102	19730	117948	45179	Alza	Alza
20180103	17416	107416	39537	Alza	Alza
20180104	20794	103288	35247	Alza	Alza
20180105	16862	102794	32463	Alza	Alza
20180106	11627	80697	30212	Alza	Alza
20180107	11482	78818	26364	Alza	Baja
20180108	16466	101844	32275	Alza	Baja
20180109	25684	110895	30061	Alza	Baja
20180110	7993	49663	13715	Alza	Alza
20180111	27535	145977	30787	Alza	Baja
20180112	16892	129676	30603	Alza	Alza
20180113	10683	98974	24951	Alza	Alza
20180114	5234	47425	13287	Alza	Baja
20180115	12498	88659	28418	Alza	Baja
20180116	23452	130139	39516	Alza	Baja
20180117	47180	185558	53776	Baja	Baja
20180118	31849	141694	35595	Alza	Alza
20180119	20283	124891	40061	Alza	Alza
20180121	7422	63703	14361	Alza	Baja
20180122	1758	13488	3069	Alza	Baja
20180123	16377	84083	23429	Alza	Alza
20180124	2110	16353	3443	Alza	Alza
20180127	7097	50042	14049	Alza	Alza
20180128	968	12003	2242	Alza	Alza
20180129	10252	96546	28357	Alza	Baja
20180131	15212	96511	35057	Alza	Baja
20180201	5892	50351	15010	Alza	Baja
20180202	4594	30808	7813	Alza	Baja
20180210	3438	59318	6391	Alza	Baja
20180211	10783	212801	22714	Alza	Baja
20180212	12332	152736	25635	Alza	Alza

20180213	7347	75772	14996	Alza	Baja
20180214	11218	101919	21634	Alza	Alza

Tabla 6.8.

Se debe observar que el análisis se hizo de forma cuantitativo y cualitativo, por lo tanto, se presentan problemas de cantidades, como por ejemplo, el spam y problemas cualitativos, como por ejemplo, el sarcasmo, la ambigüedad, el regionalismo, las oraciones gramaticalmente mal formuladas, etc.

A continuación se citan ejemplos de mala clasificación.

Falsos negativos:

- *“Those that bought Bitcoin in 2009 2010 2011 were considered as low caliber and the idiots thought it was a fraud”.*
- *“Bitcoin to the Rescue in Economically Unstable Countries <https://t.co/lGutywXFRE> <https://t.co/8N6jCl4iA9>”.*
- *“Don't Be Left Behind, Start Building Your Crypto Portfolio Today - \$ETH \$BTC \$LTC \$XRP #pAp”.*
- *“As bitcoin hits for 7 million, I want to congratulate all that didn't sell theirs during it's crash and all those tâ€”*
- *“Blockchain is good, Bitcoin is stupid.” That is the equivalent of saying "Gold is good, Gold coins are stupid because someone can make a silver coin instead." It makes no logical sense whatsoever.*
- *“Very disruptive bitcoin.”*
- *“Even if #bitcoin goes to 100k and falls down to 16k it is NEVER obsolete.”*
- *“#TheBitcoiner, A Secret #Bitcoin Millionaire Is Giving Away \$86m in Bitcoin”*
- *“Why Bitcoin Could Reach \$23,000 Soon <https://t.co/AJ38QTSyGX> #altcoin #analysis #Bitcoin”*
- *“i am an idiot who has a bitcoin from 2009 sitting in a wallet somewhere that i have no access to “*
- *“My friends were right, I'm stupid to buy bitcoins in 2010”*

Además, teniendo en cuenta que son solo dos clases, podría ser similar al accuracy de una clasificación aleatoria de 50%, esto también se debe al escaso dataset que se tiene como entrenamiento, los 20.000 tweets que se mencionaron previamente para etiquetar con GNL, tal proceso dificulta generar una correlación entre los sentimientos del tweet y la predicción de la tendencia del Bitcoin.

7. Conclusiones

El objetivo de esta tesis era el de abordar la falta de automatización en el proceso de las tendencias del Bitcoin teniendo como premisa la información que circula en Twitter.

Es por eso que el aporte principal de este trabajo consiste en una herramienta que permite inferir el sentimiento de los tweets para luego hacer un análisis cuantitativo y detectar la tendencia del bitcoin.

7.1 Limitaciones

En líneas generales, los recursos adquiridos para la construcción de la herramienta como los tweets, permitieron llevar a cabo un desarrollo exitoso de la funcionalidad propuesta. Sin embargo, se encontraron algunas limitaciones:

- Al utilizar la herramienta Google Natural Language, solo se tiene un conjunto de dataset de 20.000 tweets, las cuales resultan escasos para implementar un dataset con mejor precisión. Como se mencionó en el capítulo 3, esto es una limitante económica ya que comienza a ser pago luego de utilizar ese número de invocaciones.
- Para entrenar un clasificador que se acerque más al accuracy de GNL, se necesitaría un dataset de entrenamiento mayor e incrementar la capacidad de cómputo.
- El sarcasmo en las oraciones, dificultan la clasificación y también el proceso de entrenamiento, ya que el mismo fue automático a través de la herramienta GNL.
- La estructura gramatical en la que se encuentran los tweets, las palabras mal escritas, abreviaciones, etc.
- Variantes del precio ajenos a las noticias positivas o negativas, como por ejemplo que el capital esté fluyendo desde el bitcoin hacia otras criptodivisas alternativas.

7.2 Trabajos futuros

Como trabajo futuro, la herramienta podría escalar aumentando el tamaño del dataset para ser entrenada nuevamente y así ir cubriendo las nuevas instancias que actualmente no se pueden clasificar con Weka.

También podría implementarse una herramienta en real time asegurando disponibilidad para predecir las nuevas tendencias del Bitcoin. Crear una herramienta de estas características en real time, requiere tener gran potencia de cómputo y/o muchas instancias en paralelo. La herramienta de google tiene una latencia de un segundo por tweet analizada y en Twitter hay cinco tweets por segundo hablando sobre Bitcoin.

7.3 Implicancias

En el presente trabajo, se desarrolló un prototipo de herramienta capaz de realizar un análisis cualitativo de tweets para luego hacer uno cuantitativo y predecir la tendencia del Bitcoin, lo cual sería muy útil para las personas ahorristas, inversionistas y financieras ya que podrían adelantarse para tomar decisiones.

Para desarrollar la solución propuesta se hizo hincapié en la modificabilidad, por lo que se podría cambiar de criptomoneda a futuro y realizar la tendencias sobre otras que no sean Bitcoin.

Por otra parte, una persona consume su energía invirtiendo su tiempo en estar al tanto de las noticias de mercado de último momento y esta herramienta propone automatizar ese trabajo.

8. Anexos

8.1 Comerciantes de ruido

Da Long y otros (1990) y Sheifer (1997) presentaron un modelo basado en 'comerciantes de ruido' como lo definió Kyle (1985) para ayudar a explicar una serie de anomalías financieras, entre ellas el exceso de volatilidad de los precios de los activos y Mehra- Prescott. rompecabezas de equidad premium. Su modelo se basa en la suposición de que los inversores están sujetos a sentimientos y las apuestas contra los inversores sentimentales son riesgosas. Estos operadores de ruido pueden ser más influyentes a la hora de establecer el precio que los operadores racionales o los arbitrades. Gran parte del trabajo en torno al sentimiento de los inversores y su relación con el precio se ha basado en el trabajo de Black (1986, p. 532), quien sostiene: "El comercio de ruido se realiza mediante el ruido como si fuera información, cuanto más líquidos estén los mercados, en el sentido de tener operaciones frecuentes que nos permiten observar los precios. Pero el comercio de ruido en realidad pone ruido en los precios. "El precio de una acción refleja tanto la información con la que los comerciantes de información negocian como el ruido con el que operan los comerciantes de ruido".

El trabajo de De Long et al. (1990) ha demostrado que este ruido en el mercado influiría en el sentimiento de los inversionistas y que los inversores están sujetos a ese sentimiento. El ruido de las negras puede verse como sentimiento y los operadores de ruido intercambian sentimientos y no los fundamentos y hechos del mercado. Shleifer y Vishny later ampliaron esto (1997) mostrando los límites del arbitraje donde la alta volatilidad creada por el sentimiento del comerciante de ruido puede disuadir la actividad de arbitraje. Baker y Wurgler (2006) se han basado en este trabajo para mostrar el importante papel que el sentimiento del inversor puede desempeñar en la inserción de valores de mercado. El modelo desarrollado por Barberis y otros (1998) se basa en evidencia empírica que predice que los precios de las acciones reaccionan a patrones consistentes de buenas o malas noticias. Esto ayuda a explicar el comportamiento irracional o descontrolado de los mercados financieros durante un mercado alcista o bajista. El sentimiento del inversor se expande y, por lo tanto, influye en los precios del mercado, el inversor se negocia basándose en el sentimiento y no en los factores fundamentales.

Incluso se ha demostrado que los factores externos que afectan el estado de ánimo de los inversores en general pueden afectar los precios del mercado. Un estudio relacionó una pérdida para una nación o equipo en un evento deportivo importante, como un partido de la copa del mundo, a una caída en el mercado al día siguiente (Edmans et al., 2007). El estado de ánimo colectivo de una nación reflejado por los inversionistas y comerciantes. y su depresión reflejada en el precio de las acciones. Parece que los espíritus animales de Keynes están

trabajando. Dado que se demuestra que el sentimiento de los inversores es un factor importante que influye en los precios del mercado, el proceso de medición del sentimiento se vuelve de gran importancia.

8.2 EC2

Amazon Elastic Compute Cloud (Amazon EC2) proporciona capacidad de computación escalable en la nube de Amazon Web Services (AWS). El uso de Amazon EC2 elimina la necesidad de invertir inicialmente en hardware, de manera que puede desarrollar e implementar aplicaciones en menos tiempo. Puede usar Amazon EC2 para lanzar tantos servidores virtuales como necesite, configurar la seguridad y las redes y administrar el almacenamiento. Amazon EC2 le permite escalar hacia arriba o hacia abajo para controlar cambios en los requisitos o picos de popularidad, con lo que se reduce la necesidad de prever el tráfico.

Amazon EC2 se apoya en las tecnologías de virtualización, permitiendo utilizar gran variedad de sistemas operativos a través de sus interfaces de servicios web, personalizarlos, gestionar permisos de acceso a la red y ejecutar tantos sistemas como desee.

EC2 permite el despliegue escalable de aplicaciones proveyendo un servicio Web a través del cual un usuario puede montar una Imagen de Máquina Amazon para crear una máquina virtual, llamada por Amazon "instancia", la cual contendrá cualquier software deseado. Un usuario puede crear, lanzar y finalizar instancias de servidor tanto como necesite, pagando por hora por servidor activo, de ahí el término "elástico". EC2 provee a los usuarios control sobre localizaciones geográficas de instancias que permiten la optimización de latencia y altos niveles de redundancia.

Amazon EC2 posee una interfaz de servicios web para iniciar y configurar el servicio. Proporciona un control completo de los recursos y reduce el tiempo de arranque de sus servidores, lo que permite escalar recursos rápidamente según las necesidades del usuario. Asimismo, provee herramientas de recuperación de datos y fuerte aislamiento frente a otros procesos realizados en sus máquinas.

TeamLab AMI (Amazon Machine Image) es su propia máquina con la plataforma TeamLab preinstalada. Se ofrece gratis como una parte del servicio Amazon EC2 que permite ejecutar una instancia (máquina virtual) en los servicios seguros de la nube de Amazon.

8.3 Putty

PuTTY es un emulador gratuito de terminal que soporta SSH y muchos otros protocolos. La mayoría de usuarios, especialmente los que trabajan sobre sistemas operativos Windows, lo encuentran muy útil a la hora de conectar a un servidor Unix o Linux a través de SSH.

PuTTY ofrece una interfaz gráfica de configuración muy sencilla e integra múltiples opciones:

- Guardar las preferencias de conexión para establecerla rápidamente en el futuro

- Respuestas de puertos

- Soporte Ipv6

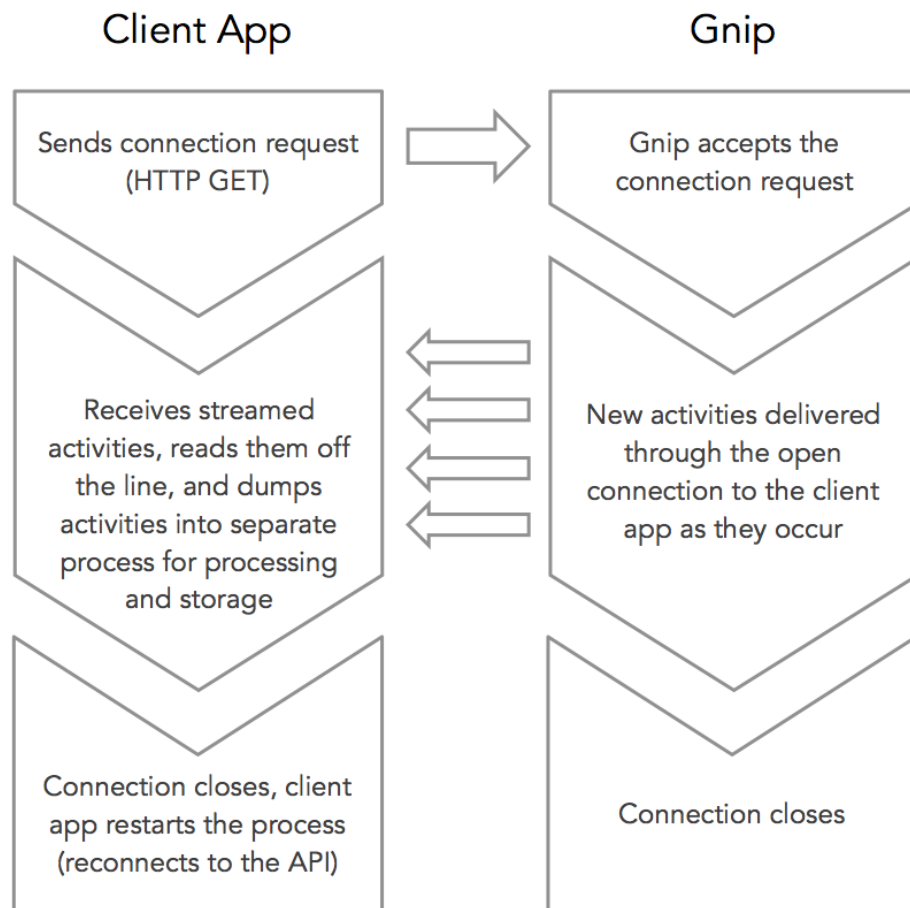
- Soporte SCP y SFTP

Además de ser para Windows, PuTTY ha sido portado a sistema Linux y otros sistemas operativos con núcleo Unix, y por si fuera poco, pronto veremos una versión para Mac OS X. Como muchísimas aplicaciones multiplataforma, PuTTY es de código abierto y se distribuye bajo licencia MIT.

8.4 Listener

PowerTrack, Volume (p. Ej., Decahose, Firehose) y Replay utilizan el protocolo HTTP de transmisión para entregar datos a través de una conexión abierta de API de transmisión. En lugar de entregar datos en lotes a través de solicitudes repetidas de su aplicación cliente, como podría esperarse de una API REST, se abre una única conexión entre su aplicación y la API, y se envían nuevos resultados a través de esa conexión cada vez que se producen nuevas coincidencias. Esto da como resultado un mecanismo de entrega de baja latencia que puede soportar un rendimiento muy alto.

La interacción general entre su aplicación y la API de Twitter para estas transmisiones se describe a continuación:



Referencias

Kwak, H., Lee, C., Park, H., & Moon, S. (2010, April). What is Twitter, a social network or a news media?. In Proceedings of the 19th international conference on World wide web (pp. 591-600). AcM.

Ron, D., & Shamir, A. (2013, April). Quantitative analysis of the full bitcoin transaction graph. In International Conference on Financial Cryptography and Data Security (pp. 6-24). Springer, Berlin, Heidelberg.

Olivera, J. H. (1965). Inflación estructural y política financiera. *Ola Financiera*, 5(12).

Sebastiani, F. (2002). Machine learning in automated text categorization. *ACM computing surveys (CSUR)*, 34(1), 1-47.

Nasrabadi, N. M. (2007). Pattern recognition and machine learning. *Journal of electronic imaging*, 16(4), 049901.

Nair, B. B., & Mohandas, V. P. (2015). Artificial intelligence applications in financial forecasting—a survey and some empirical results. *Intelligent Decision Technologies*, 9(2), 99-140

Cofnas, A. (2016). Trading binary options: strategies and tactics. John Wiley & Sons.

Kouloumpis, E., Wilson, T., & Moore, J. D. (2011). Twitter sentiment analysis: The good the bad and the omg!. *Icwsn*, 11(538-541), 164.

Witten, I. H., Frank, E., Trigg, L. E., Hall, M. A., Holmes, G., & Cunningham, S. J. (1999). Weka: Practical machine learning tools and techniques with Java implementations.

Deltell, L., Claes, F., & Osteso, J. M. (2013). Predicción de tendencia política por Twitter: Elecciones andaluzas 2012. *Ámbitos. Revista Internacional de Comunicación*, (22).

Herrero Amorós, J. (2018). Modelado predictivo de la dirección de la cotización del Bitcoin utilizando índices de mercado, análisis de sentimientos en Twitter e índices de popularidad por término mediante Google Trends.